



软件开发技术

第9章 面向对象分析技术

浙江大学 机械工程学院 航空制造工程研究所

王青 李江雄

1、建立类模型

1.1 分析类

1.2 一种通用的类模型建立方法

1.3 类建模实例

2、领域状态模型

3、领域交互模型

4、将分析迭代



01

建立类模型

◆ 分析的目的

通过调查以及和用户讨论，对所要解决的问题进行详细定义，并加以确切地描述。

◆ 在面向对象技术中，对问题和软件的描述都采用：

类模型、状态模型和交互模型

◆ 面向对象分析的目的是：理解由问题陈述所描述的真实世界的系统，并把它的功能抽象为分析模型（类模型，状态模型和交互模型）。

- 类模型描述对象的静态结构
- 状态模型描述对象的生存期
- 交互模型描述对象之间的交互

◆ 面向对象的分析过程可以划分为两个阶段：

- 领域分析：专注于理解问题的本质
- 应用分析：在领域模型的基础上合并用户可见的与应用程序相关的内容

- ◆ 领域类模型主要描述真实世界的类以及它们之间的相互关系。
- ◆ 此过程中，更加关注类模型。
- ◆ 为什么要建立类模型？
 - 类是建立面向对象模型的核心，没有类就没有对象。
 - 类模型是后续建立状态模型和交互模型的基础。
 - 类模型即类和类之间的关联提供了系统的总体结构和方法
- ◆ 领域类模型分析的步骤：
 - 先寻找类和关联
 - 再增加属性，用来描述类和关联的基本网络
 - 最后使用继承和聚合来组织这些类

1.1 分析类方法：

官方文档的定义：分析类用于获取系统中**主要的“职责簇”**。它们代表系统的**原型类**，是系统必须处理的主要抽象概念的**第一个关口**，如果期望获得系统的高级概念性描述，则可对分析类进行维护。分析类还可产生系统设计的主要抽象-系统的设计类和子系统。

从定义可知：

- ◆ 分析类代表系统中主要的职责簇，这意味着分析类是**从功能性需求**向计算机实现转化过程中的第一个关口；
- ◆ 分析类可以产生系统的设计类和子系统，这意味着**计算机实现是可以通过某种途径“产生”出来的**，而不是拍脑袋拍出来的。

上述两点决定了分析类是跨越需求到设计实现的桥梁，是从业务需求向系统设计转化过程中最为主要的元素，它们在高层次抽象出系统实现业务需求的原型，业务需求通过分析类逻辑化，被计算机所理解。

分析类包括3种类型：**边界类、控制类和实体类**。

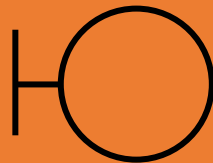
□ 边界类

边界类（boundary class）是一种用于对系统外部环境与其内部运作之间的交互进行建模的类。对现实世界来说，边界类的实例可以是窗口、通信协议、打印机接口、传感器、终端等。在计算机世界里，边界类也可以是一个消息中间件、一个驱动程序、一组对象接口等。如何从用例图中找出边界类？

- ◆ 参与者与用例之间应当建立边界类，将用例可以提供给参与者完成业务目标的操作通过边界类暴露出来；
- ◆ 用例与用例之间如果有交互，应当为其建立边界类。一个用例如果要访问另一个用例，直接访问用例内部对象是不好的结构，将导致紧耦合的发生。使用边界类可以隔离直接访问，其作用相当于代理模式；
- ◆ 如果用例与系统边界之外的非人对象有交互，例如第三方系统，应当为其建立边界类，专门负责与第三方系统进行交互。

从架构角度上说，边界类主要位于交互层。

边界类



□ 控制类

控制类（control class）用于对一个或几个用例所特有的控制行为进行建模。控制对象（控制类的实例）通常控制其他对象，因此它们的行为具有协调性质。控制类源于对用例场景中**动词的分析和定义**，包括限制动词的描述。

例如：寄信人到邮局寄信的用例场景，该场景可以描述为：寄信人到达邮局后，购买信封，将信装入信封，写上地址，称重，计算邮资，购买邮票，邮寄信件，拿走回执。在这个场景中，购买、装入、写上、计算、购买、贴上、邮寄等行为都可以成为控制类的来源。

应强制使用控制类，这是一种好的程序结构。如果边界类直接访问实体类，容易造成业务逻辑代码要么与显示混在一起，要么与数据逻辑混在一起，代码结构混乱，易于出错，不易修改。因而应当养成在**边界类和边界类、边界类和实体类、实体类和实体类之间都默认加入控制类，将相关的处理逻辑放到控制类中，哪怕控制类只有一个操作。**

□ 实体类

实体类 (entity class)是用于对必须存储的信息和相关行为建模的类。实体对象（实体类的实例）用于保存和更新一些现象的有关信息，例如事件、人员或者一些现实生活中的对象。实体类通常都是永久的，它们所具有的属性 and 关系是长期需要的，有时甚至在系统的整个生存周期都需要。

实体类源于业务模型中的业务实体，很多时候可以直接把业务实体转化为实体类，例如寄信人到邮局寄信业务模型的业务实体就可以直接把信、信封、邮票、信件、回执等抽象为实体类。但出于系统结构优化的需要，一些业务实体可以在后续的过程中被分拆、合并。

例如：邮局寄信业务实体中，地址是信封的一个属性，为满足打印地址清单、按地址分发信件等需要，可以单独处理地址将其变为实体类，这样原来信封→信件之间一对一的关系，可以重新通过地址实体类建立如下关系：

地址¹→¹信封；地址¹→¹信件

电梯问题：

该问题的逻辑原理是：满足以下约束条件在 m 个楼层之间移动 n 个电梯：

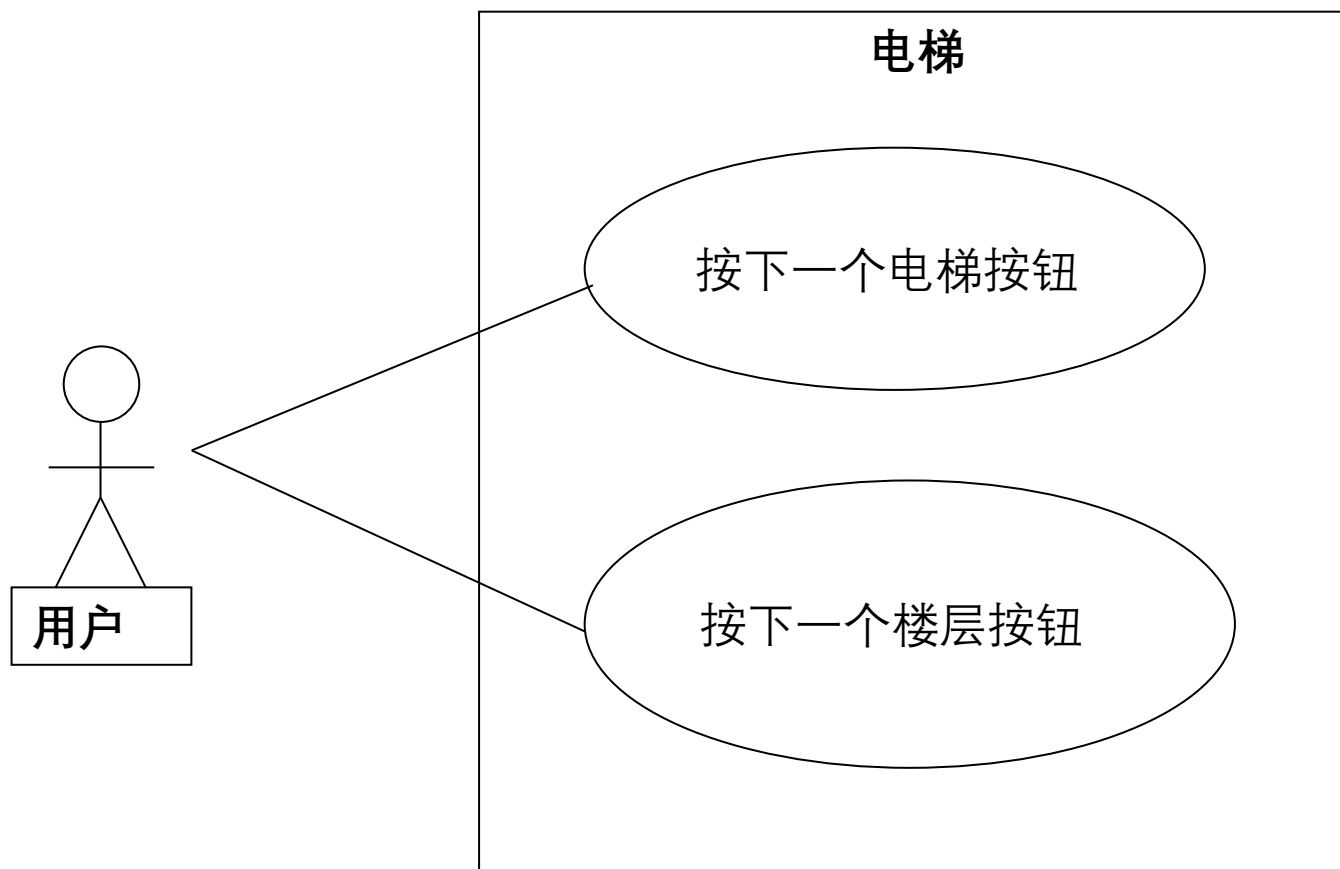
- 1) 每个电梯有 m 个按钮(button)，每个按钮对应一层。当有人按下按钮时，按钮变亮并指示电梯到相应的楼层。当该电梯到达相应的楼层时，按钮变暗。
- 2) 除了一楼和顶层外，每层有两个按钮，一个请求电梯向上，一个请求电梯向下。按钮按下时变亮。当一个电梯到达该层并往请求方向移动的时候，按钮将变暗。
- 3) 当一个电梯没有请求时，它停留在当前楼层，电梯门关闭。

问题中有两组按钮。在 n 个电梯中，每个电梯有 m 个按钮，每个按钮对应一层。因为这 $n \times m$ 个按钮在电梯里面，称为电梯按钮。每个楼层有2个按钮，一个请求电梯向上，一个请求电梯向下，称为楼层按钮。每一个按钮可处于两个状态之一：开或关



1 建立类模型

□ 电梯问题功能建模:



电梯问题案例研究用例图

一个正常场景：

1. 用户A在3楼按下向上的楼层按钮，请求一部电梯的服务。用户A想上7楼
2. 向上的楼层按钮被打开
3. 一部电梯到达3楼。电梯里有用户B，他是在1楼进入电梯的，并按下了去9楼的电梯按钮
4. 电梯门打开
5. 定时器开始计时 (用户A进入电梯)
6. 用户按下去7楼的电梯按钮
7. 去7楼的电梯按钮被打开
8. 电梯门在超时后关闭
9. 向上的楼层按钮被关上
10. 电梯移动到7楼
11. 去7楼的电梯按钮被关上
12. 电梯门打开
13. 定时器开始计时 (用户A离开电梯)
14. 电梯门在超时后关闭
15. 电梯与用户B向9楼移动

□ 实体类建模

➤ 名词提取法：

一种确定实体类的方法是从用例推导出实体类。开发人员仔细研究所有场景，包括正常的和异常的，并找出用例中起作用的组件。

识别问题域中的实体，实体的描述通常用名词、名词短语、名词性代词的形式出现。

用指定语言对系统进行描述：描述过程应与领域专家共同合作完成，并遵循问题域中的概念和命名。

从系统描述中标识名词、名词短语、名词性代词：为了发现对象和类，开发人员要在系统需求和系统分析的文档中查找名词和名词短语，包括：可感知的事物、角色、事件、互相作用、人员、场所、组织、设备和地点等。

从电梯正常操作的场景中可以发现电梯按钮、楼层按钮、电梯、门和定时器。一般情况下，候选实体类跟在实体类建模期间提取出的实际类是很接近的。

➤ CRC卡片法:

类-职责-协作 (class-responsibility-collaboration, CRC) 卡片用于分析 workflow, 每张卡片分成3部分, 包括该类的名称、功能 (职责) 和被它调用以完成其功能的一组类 (协作)。CRC卡片通常直接包含类的属性和方法, 以连线表示协作关系。

CRC卡片方法提供了一种简单标识和组织与系统或产品需求相关的类的手段, 一旦系统的基本使用场景 (用例) 确定后, 则要标识候选类, 指明它们的责任和协作, 即类-责任-协作者建模。

类名:	
职责:	协作者:

名词提取法建模：

□ 阶段1：用一段话描述软件产品

对电梯控制问题可描述为：电梯里和楼层的按钮控制一幢m层大楼里的n个电梯移动。在某楼层按下请求按钮时，按钮变亮，当该要求满足时，发亮中断。当一个电梯没有请求时，它停留在当前层，电梯门关闭。

□ 阶段2：识别名词

先从中识别出在系统之内的名词，然后将这些名词作为候选的实体类。

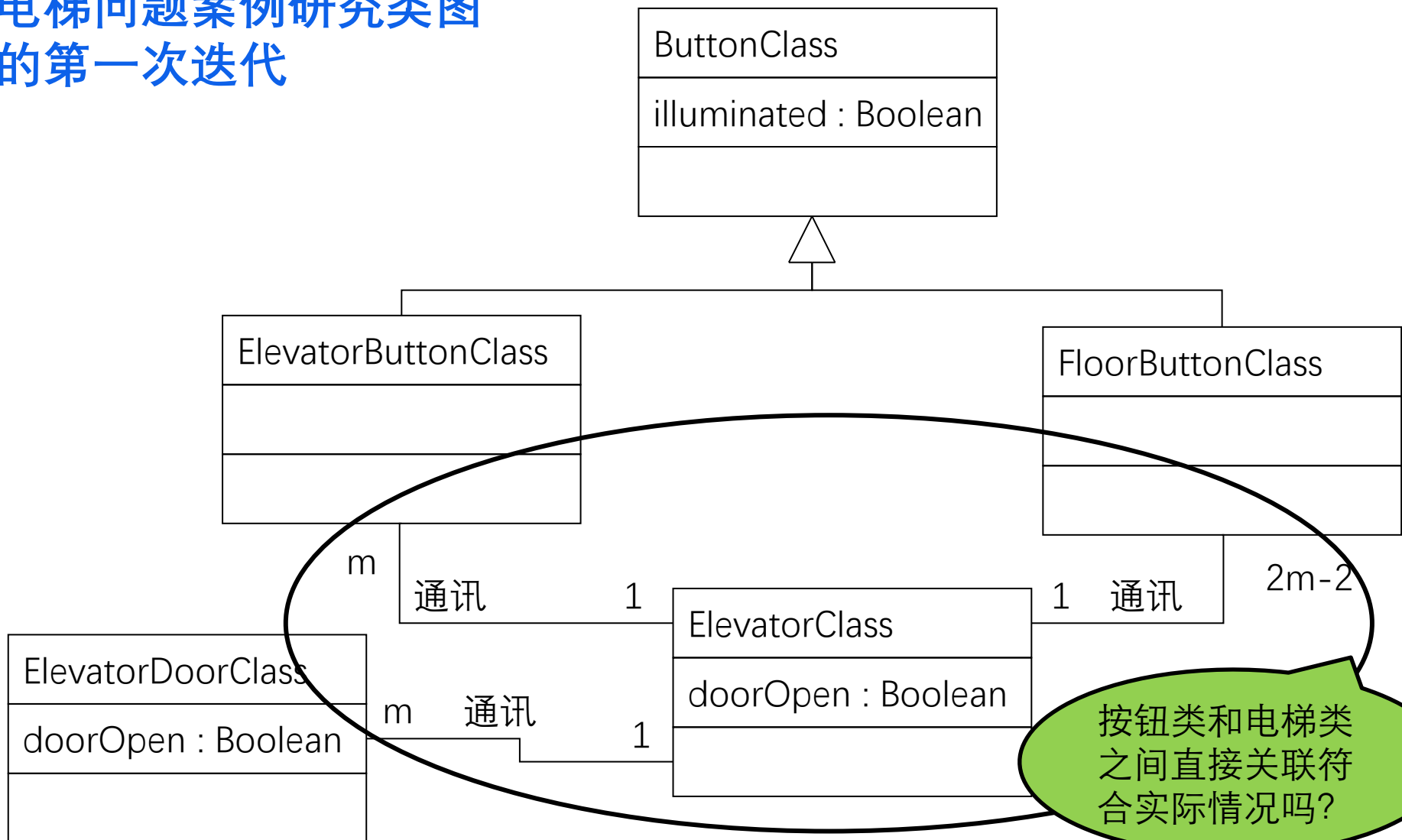
电梯里和楼层的按钮控制一幢m层大楼里的n个电梯移动。在某楼层按下请求按钮时，按钮变亮，当该要求满足时，发亮中断。当一个电梯没有请求时，它停留在当前层，电梯门关闭。

共8个名词，其中楼层、大楼在问题边界之外，移动、发亮、请求是抽象名词，没有物理存在的事物。一个实用的经验法则：抽象名词很少是类，一般为类的属性，例如，发亮是按钮的属性。因而剩下的三个名词为候选实体类：电梯类（Elevator class）、电梯门类（ElevatorDoor class）和按钮类（Button class）。



1 建立类模型

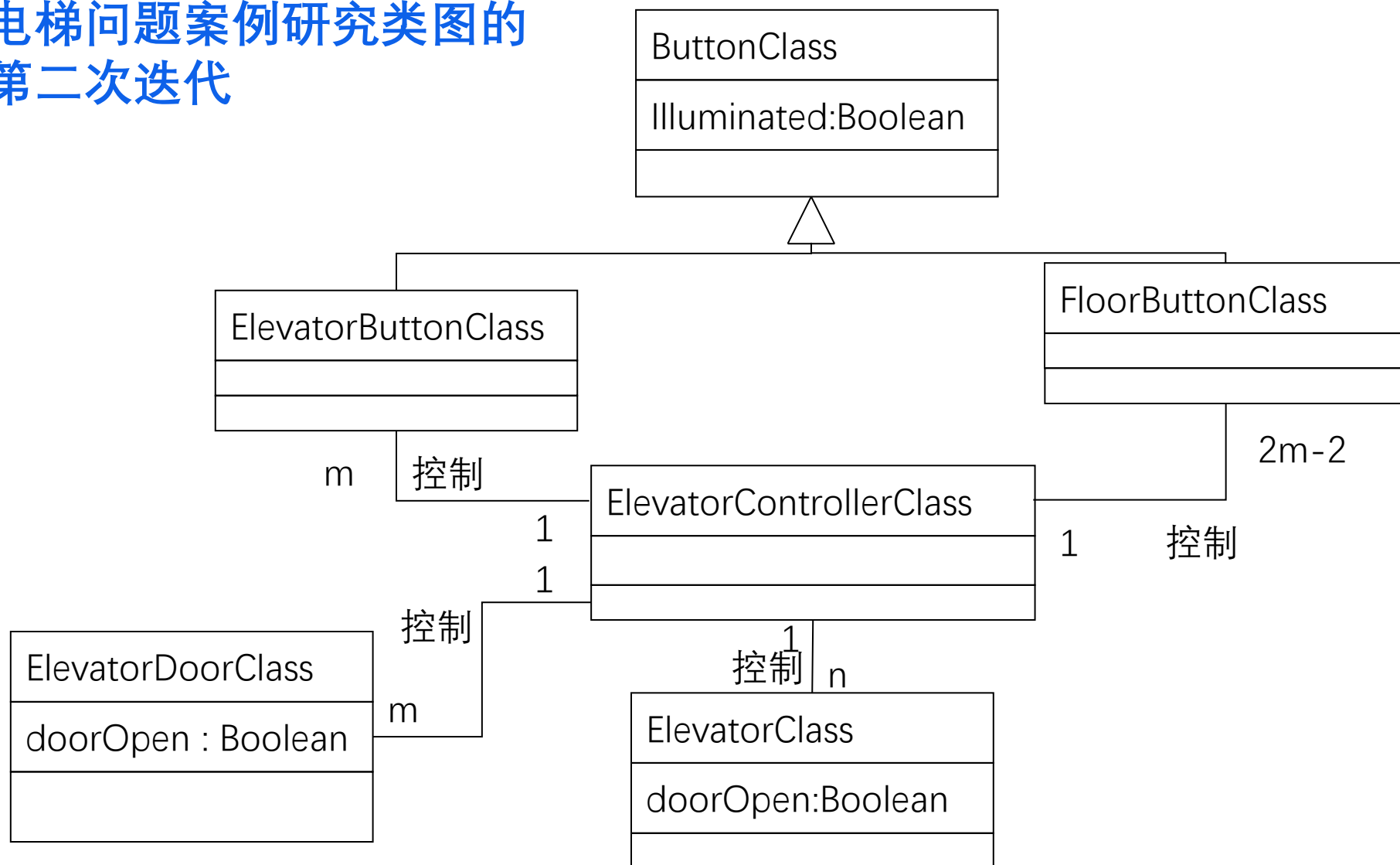
电梯问题案例研究类图 的第一次迭代





1 建立类模型

电梯问题案例研究类图的第二次迭代





1.2 一种通用的类模型建立方法

建立类模型具体步骤：

- (1) 类的识别
- (2) 抛弃不必要和不正确的类
- (3) 寻找类间关联
- (4) 抛弃不必要和不正确的关联
- (5) 定义关联终端名、限定关联、多重性
- (6) 寻找属性
- (7) 删除那些不必要和不正确的属性
- (8) 使用继承来组织类结构

■ (1) 类的识别

创建类模型的第一步就是从应用领域寻找相关的对象类。

类的识别是面向对象方法的一个难点，但又是建模的关键。常用的方法有：

- 1) 名词识别法
- 2) 系统实体识别法
- 3) 从用例中识别类
- 4) 利用分解与抽象技术

□ 系统实体识别法

不关心系统的运作流程及实体之间的通信状态，而只考虑系统中的人员、组织、地点、表格、报告等实体，经过分析将它们识别为类（或对象）。

被标识的实体有：

- 系统需要存储、分析、处理的信息实体；如CAD系统中的点、线、面等几何元素
- 系统内部需要处理的设备；如运动控制系统中的被控制设备
- 与系统交互的外部系统；可建立外部系统的代理类，所有对外部系统的交互通过代理类进行。
- 系统相关人员；如教务管理系统中的教师、学生
- 系统的组织实体。如公司业务管理系统中的部门、项目组等

例：有一个超市，顾客可在货架上自由挑选商品，由收款机收款，收款机读取商品上的条形码标签，并计算商品价格。收款机应保留所有交易的记录，以备账务复查及汇总使用。

通过分析问题的陈述，确定以下几类实体：

- (1) 信息实体：交易记录、商品、税务信息、销售记录、库存记录。
- (2) 设备：收款机、条形码扫描器。
- (3) 交互系统：信用卡付款系统。
- (4) 人员职责：收款员、顾客、会计、经理。
- (5) 系统的组织实体：本例不考虑。

以上列出的实体，都可以直接识别为类。



□ 从用例中识别类

根据用例的描述来识别类：

- 用例的描述中出现哪些实体？
- 用例执行过程中产生并存储哪些信息？
- 与用例关联的角色向用例输入什么信息？
- 用例又向该角色输出哪些信息？

□ 分解与抽象技术

在确定类时，常使用两类技术：

(1)分解技术 将整体类和组合类分解，这样可以控制单个类的规模。

(2)抽象技术 根据一些类的相似性建立抽象类，并建立抽象类与这些类之间的继承关系。

抽象类实现了系统内部的重用，很好地控制了复杂性，并为所有子类定义了一个公共的接口，使设计局部化，提高系统的可修改性和可维护性。

■ (2) 抛弃不必要和不正确的类

- 去掉冗余类：如两个类表述同一信息，应保留最具有描述能力的类。
- 去掉不相干的类：删除与问题无关或关系不大的类。
- 删除模糊的类或性质独立性不强的类：有些初始类边界定义不确切，或范围太广，应该删除。
- 所描述的是属性，不适宜作为对象类；
- 所描述的只是实现过程中的暂时的对象，应删去。

◆ 甄别操作与类：

- 如果某名称描述的操作可以应用于对象，但不可以被独立使用，那么他就不是类。例如，打电话是涉及呼叫者和电话网络的一系列动作。如果只是要创建电话，那么CALL就是状态模型的一部分，而不是类。
- 拥有许多自身特征的操作应该被建模成类。例如，在电话呼出统计系统中，CALL是一个重要的类，它拥有诸如date, time, origin 和Destination等属性。

◆ 甄别角色与类：

类的名称应该反映其本质特征，而不是反映它在关联中所扮演的角色。

例如，对于汽车制造商数据库的类来说，OWNER这个名字就不太好。如果要添加一组司机那会怎么样？租赁汽车的人又该如何？正确的类应该是Person（或者也可能是Customer），它可以充当不同的角色，例如Owner, Driver和Lessee。

◆ 抛弃已实现的类：

清理分析模型中那些与真实世界无关的类，在设计阶段会需要它们，但现在却不需要。

例如，尽管CPU、子程序、进程、算法和中断对于操作系统来说是合法类，但对于大多数应用而言它们却都是已实现的类。

数据结构，例如链表、树、数组和表，几乎也总是已实现的类。

问题陈述：

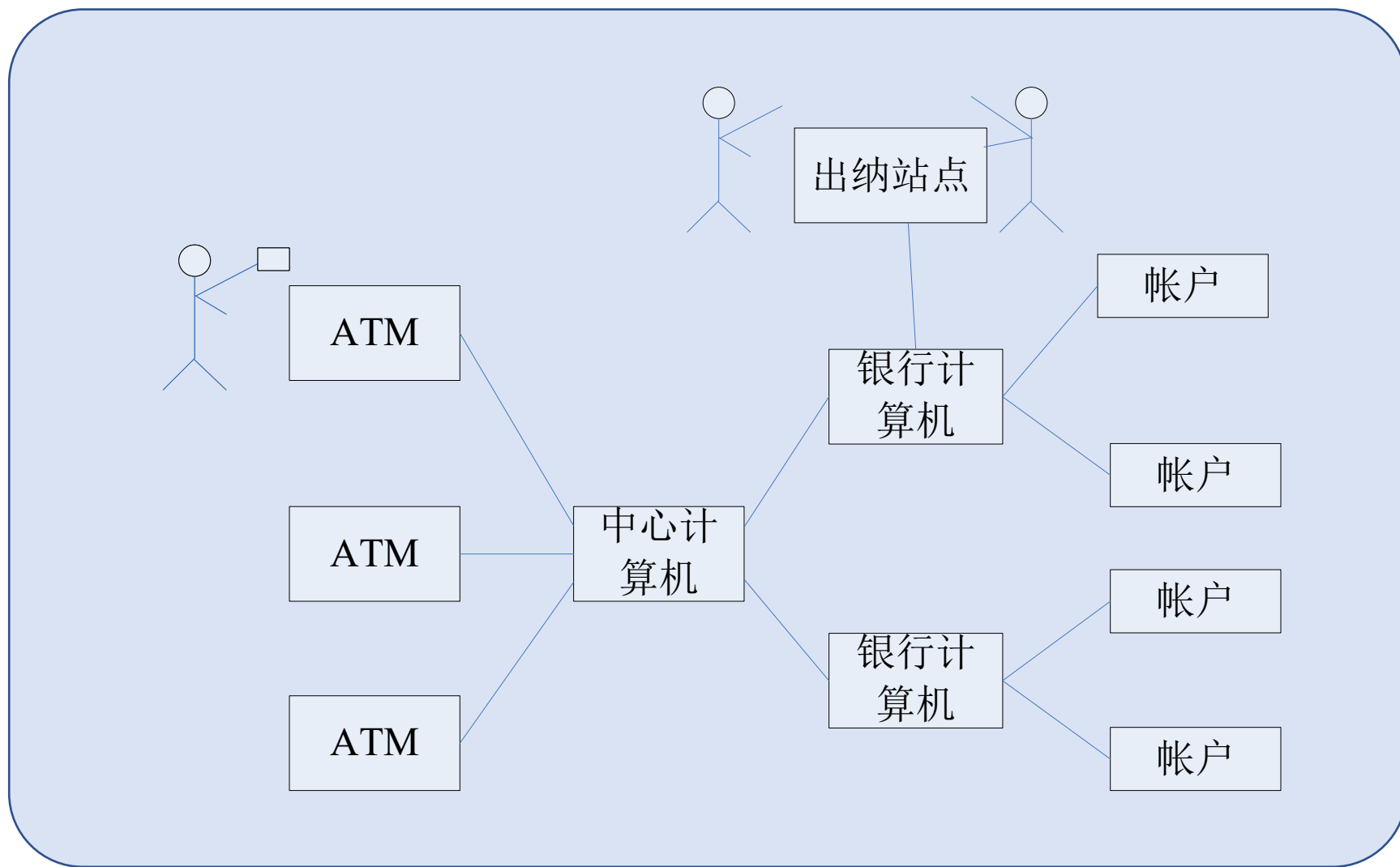
设计软件以支持计算机管理的银行网络，网络既要包括出纳站点，又要包括为银行协会内部所共享的自动取款机。每一家银行（分行、分理处）都提供自己的计算机来维护帐户，处理账户交易事务。

出纳站点由各个银行所拥有，它们能够使用银行自己的计算机直接进行通信，由出纳员输入帐户信息和事务数据。

自动取款机要与中心计算机通信，与相应的银行交换交易清帐信息。自动取款机的功能包括接受现金卡、与用户交互、与中心系统通信以执行交易（发放现金）、退出现金卡和打印收据。系统要保存有适当的记录，而且还需要安全措施。系统必须能够正确地处理对同一帐户的并发访问。网络费用平均摊派给各分理处。

其网络图如下页：

银行网络管理网络图



初始寻找的类

从问题陈述名词中提取得到初始类：

软件	银行网络	出纳站点	分行	系统	自动取款机
账户	账户数据	出纳员	分理处	访问	分理处计算机
事务	事务数据	现金卡	用户	顾客	分行计算机
现金	收据	网络费用		安全措施	记录保管

抛弃不必要和不正确的类

- **去掉冗余类**：如两个类表述同一信息，应保留最具有描述能力的类，如“用户”与“顾客”是重复的描述，由于“顾客”更具有描述性，故保留它，删除“用户”。
- **去掉不相干的类**：费用（Cost）就位于银行网络管理软件范畴之外，它是银行之间的事。

采用名词识别法：检查问题陈述中的所有名词，得到初始类：

软件	银行网络	出纳员	分行	系统	自动取款机
账户	账户数据	出纳站	分理处	访问	分理处计算机
事务	事务数据	现金卡	用户	顾客	分行计算机
现金	收据	网络费用	安全措施		记录保管

- **删除模糊的类**：有些初始类边界定义不确切，或范围太广，应该删除。如“软件”、“系统”、“安全措施”、“记录保管”、“银行网络”。
- **删除那些性质独立性不强的，而应该是类“属性”的候选类**：如“账户数据”、“收据”、“现金”、“事务数据”。
- **所描述的操作不适宜作为对象类**，并被其自身所操纵，所描述的只是实现过程中的暂时的对象，应删去，如“访问”。

采用名词识别法：检查问题陈述中的所有名词，得到初始类：

软件	银行网络	出纳员	分行	系统	自动取款机
账户	账户数据	出纳站	分理处	访问	分理处计算机
事务	事务数据	现金卡	用户	顾客	分行计算机
现金	收据	网络费用	安全措施	记录保管	

要从ATM示例中清除的类：

模糊类

软件
系统
安全措施
记录保管
银行网络

属性

账户数据
收据
现金
事务数据

冗余类：

用户

实现：

访问

不相干类：

费用

好类候选（11个）

分行计算机 分行 出纳站
出纳员 分理处计算机
自动取款机 账户 现金卡
事务 顾客

■ (3) 寻找类之间的关联

关联的基本概念

1) 、链接 (link)和关联(association)

链接用来建立对象之间的关系； **关联**用来建立类之间的关系

□两个对象之间有相互关系，称为两者之间有链接；链接是对象之间物理上或概念上的连接。如李某某为浙江大学工作，这里为...工作是一个链接。大多数链接联系两个对象。

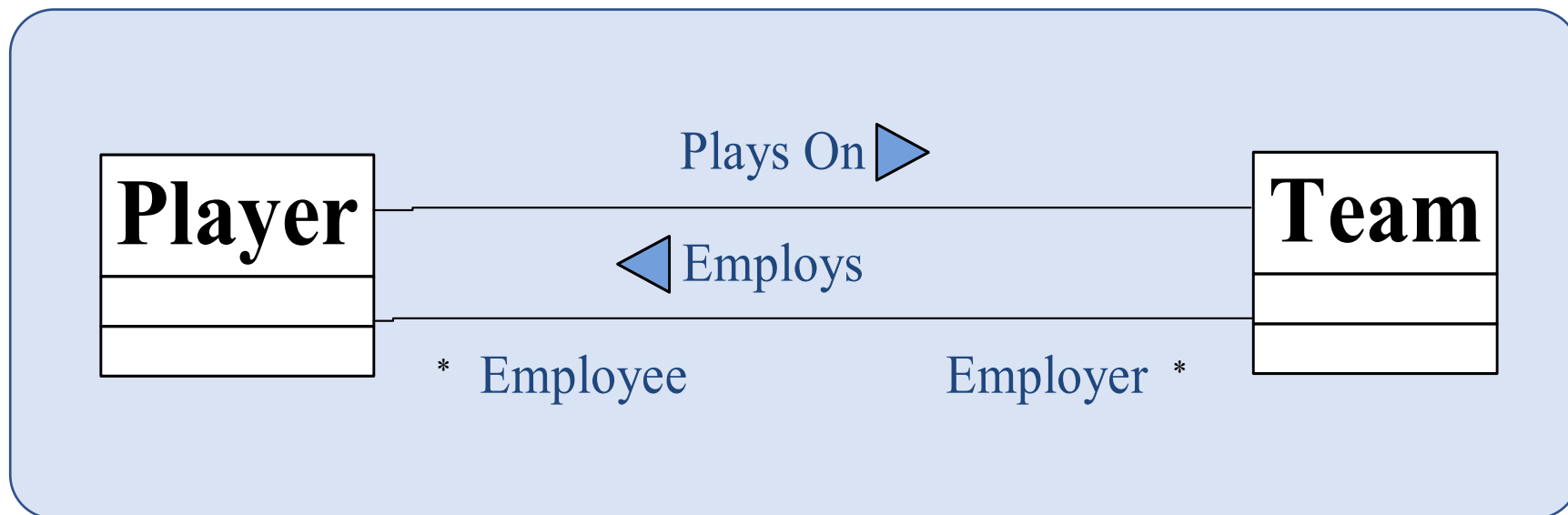
□**关联**描述有着共同的结构和语义的一组链接，需要有名字。如人为公司工作，这里为...工作作为一个关联。

关联与链接的关系就象类与对象的关系一样，以下为关联的一些例子：

- 职业运动员为球队打球，球队雇佣职业运动员
- 教师被学校雇佣
- 银行出纳为顾客服务

2) 、角色

由于当一个类与另一个类发生关联时，每个类通常在关联中都扮演某种**角色**，可以为角色分配一个名字，称之为关联终端名。



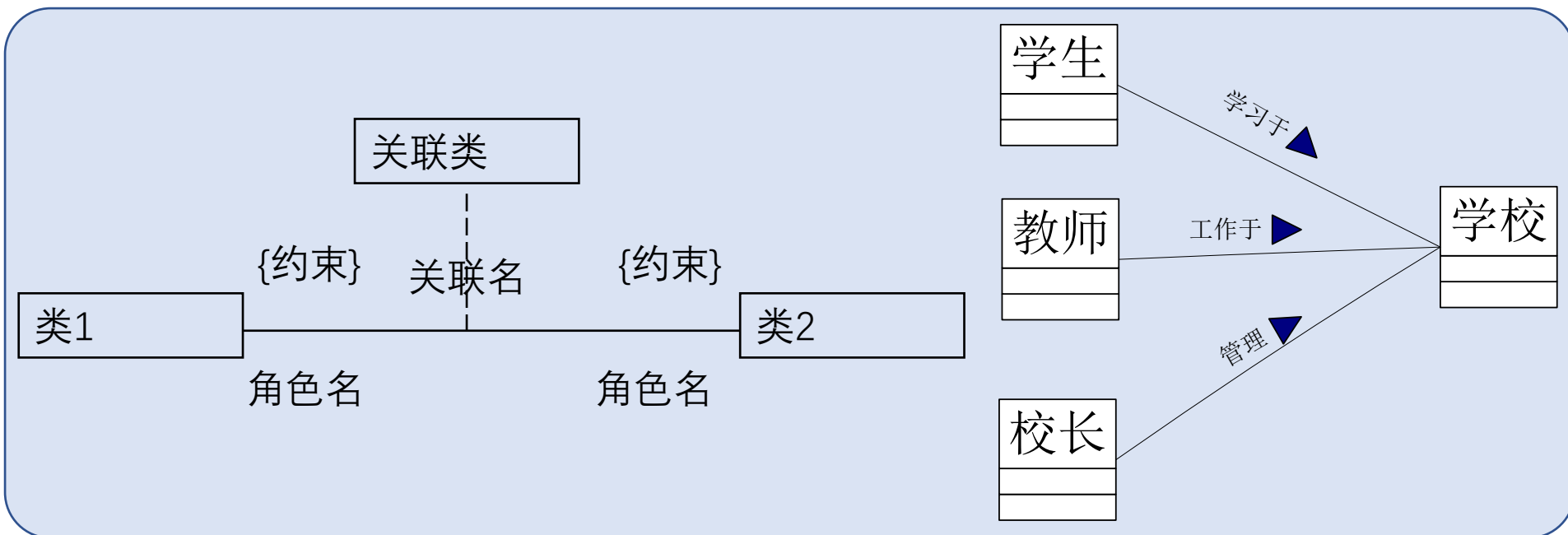
3)、关联类

关联也可以有自己的属性和操作，那么也就可以将关联建模为一个类，这种描述关联的类，称为关联类。

4)、关联上的约束

关联上的约束定义了关联的规则，如必需按顺序进行关联、两个关联取其一

5)、关联的UML表达



□ 关联的多重性 (multiplicity)

多重性： 指定一个类与另一个类的单个实例可能相关的实例数目。它约束了相关对象的数目。

多重性的常见情况为：

- 1对1;
- 1对多 ;
- 1对“至少m 最多 n”;
- 1对“0或1”
- 1对“1或多”;
- 1对“1组中某一个数”

UML中的符号：

- * 多
- 1..* 1或多
- m, n m个或者n个
- 0..1 0或1
- m..n 至少m个，最多n个

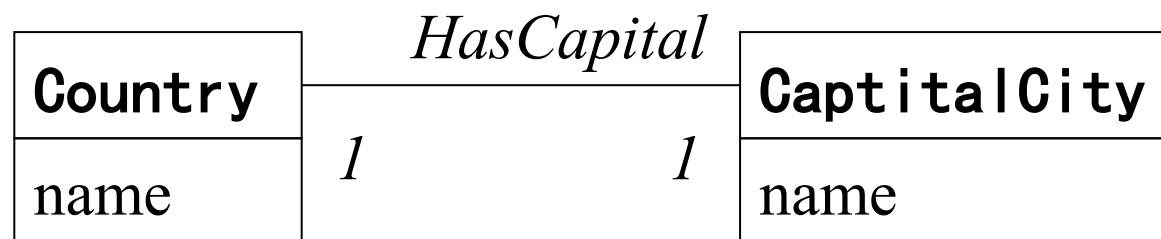


1 建立类模型

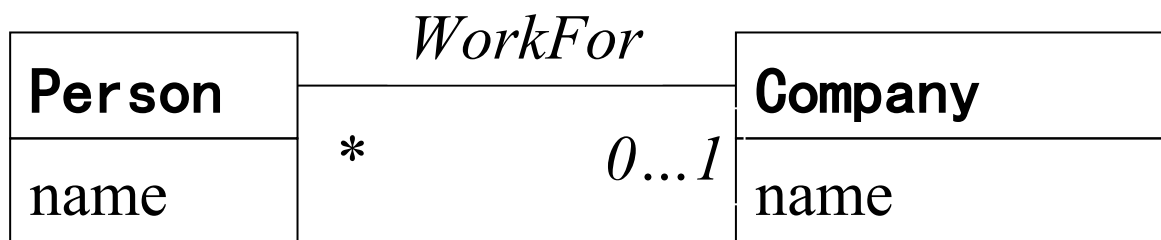
关联及多重性的例子



一个人，可以有多个公司股票。
一个公司可以有多个个人持有股票

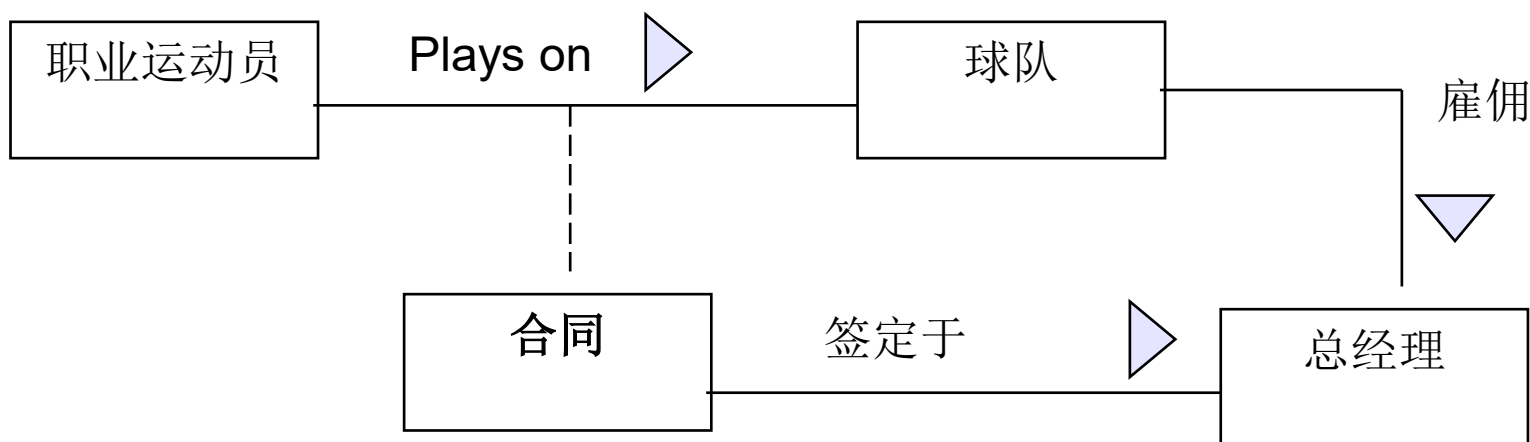


一个国家有一个首都，一个首都只属于一个国家



一个人，可以没有工作，也可以在一个公司工作。
一个公司可以有多个员工

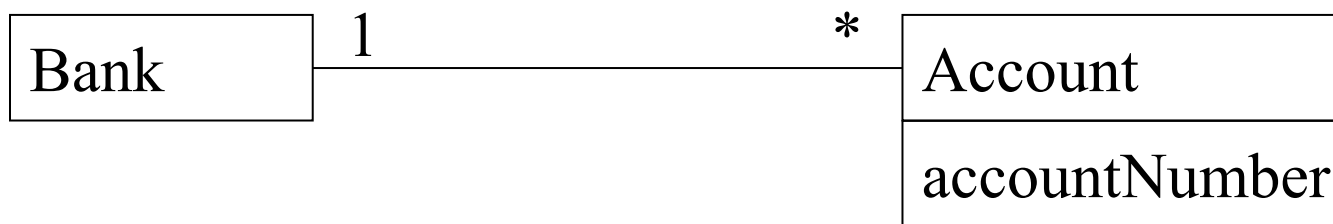
关联类的图例：



1 建立类模型

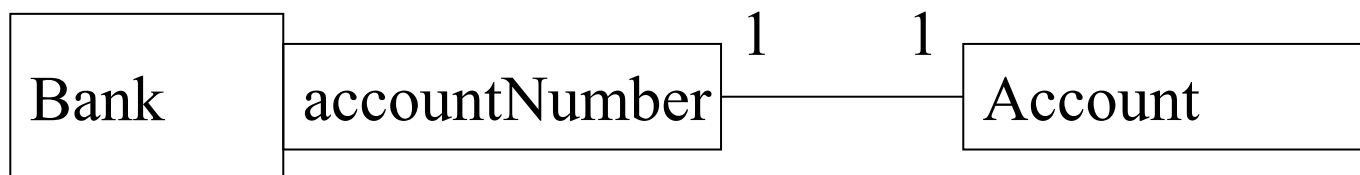
□ 限定关联及其UML表示

限定关联是这样一种关联，其中被称为限定符的属性会消除在关联“多”端上的对象的歧义。限定符在目标对象之间进行选择，将有效的多重性从多降为1。目标多重性为1或0..1的限定关联确定了一条准确的路径，可以从源对象找到目标对象。



未限定

一个银行有多个帐户。一个帐户有一个银行

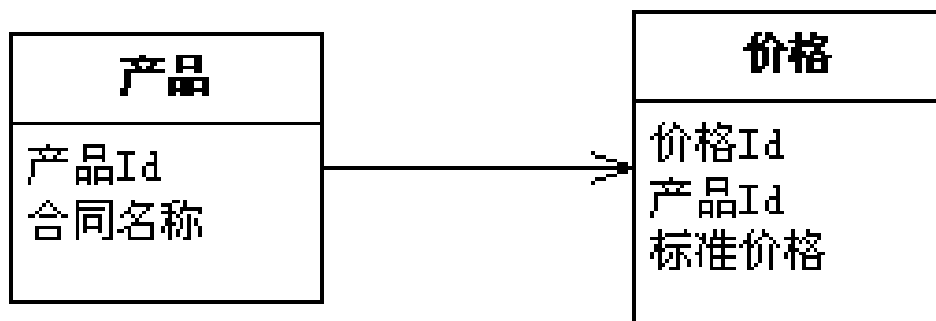


限定

在银行环境中，帐号确定唯一的帐户

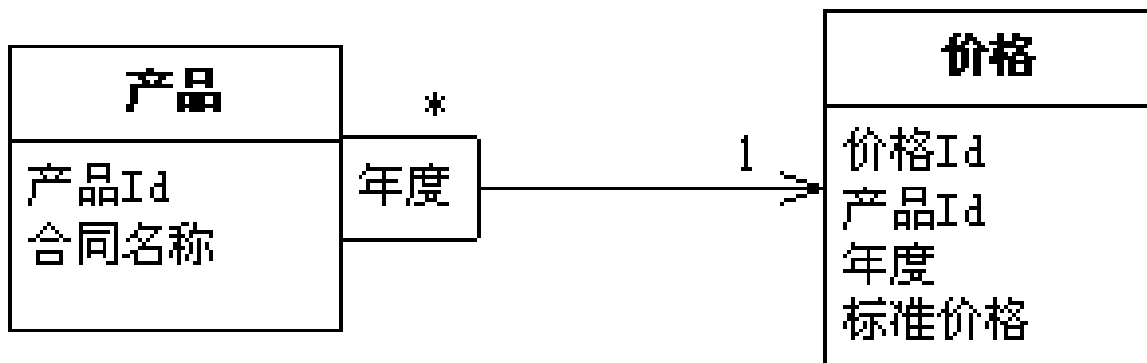
限定符的例子：

领域知识：一种产品有一个价格。



但是，这里没有表现出“一种产品每个年度只有一个价格”这个领域知识。在这种情况下，可以使用关系限定符来更精确地表达产品和价格的关系。

新的领域知识：每年要有一套价格表



□ 寻找类间关联

- 两个或多个类之间的结构化关系就是关联，从一个类到另一个类的引用也是关联。
- 属性不应该引用类，要使用关联。例如，类Person不应该有属性Employer；类Person和类Company之间要使用关联Works For联系。关联在与类自身相同的抽象层次上显示类间的关系，而对象值属性则隐藏了依赖，模糊了它们的双向本质。可以使用不同的方式实现关联，但这种实现决策应该保持在分析模型之外，以保留设计的自由。
- 关联常常与静态动词或动词短语相对应，这些动词或短语包括：
 - ◆ 物理位置 (Next To, Part Of、Contained in)
 - ◆ 定向动作
 - ◆ 通信
 - ◆ 拥有者关系

从问题陈述中提取所有的候选项，先把它们记在纸上；不要过早地提炼内容。再说一次，不要盲目对待语法形式；无论用什么语言来表达，目标都是要捕获关系。

银行网络系统问题陈述中的关联

- ◆ 银行网络包括出纳站和自动取款机
- ◆ 分行共享自动取款机
- ◆ 分理处提供分理处计算机
- ◆ 分理处计算机保存账户
- ◆ 分理处计算机处理账户支付事务
- ◆ 分理处拥有出纳站
- ◆ 出纳站与分行计算机通信
- ◆ 出纳员为账户录入事务
- ◆ 自动取款机接受现金卡
- ◆ 自动取款机与用户接口
- ◆ 自动取款机发放现金
- ◆ 自动取款机打印收据
- ◆ 系统处理并发访问
- ◆ 分理处提供软件
- ◆ 费用分摊给分理处

隐含动作短语

- 分行由分理处组成
- 分理处拥有账户
- 分行拥有分行计算机
- 系统提供记录保管
- 系统提供安全
- 顾客有现金卡

问题领域的知识

- 现金卡访问帐户
- 银行雇用出纳员



■ (4) 抛弃不必要和不正确的关联

具有下列特性的候选项一般是不必要的，或不正确的关联。

□被删除类之间的关联

如果删除了关联中的一个类，那就必须要删除该关联或者在用其他类来描述它。银行网络管理示例中可以删除关联：分理处拥有出纳柜台和ATM机的[银行网络](#)、ATM打印[收据](#)、银行提供[软件](#)、[费用](#)分摊到银行、系统提供[日志记录](#)和系统提供[安全功能](#)。

□无关关联或实现关联

删除位于问题领域之外或处理实现制品的任何关联。银行网络管理示例中，[系统处理并发访问](#)是一个实现概念。真实世界的对象本质上都是并发的；访问算法也都要实现成并发的。

□表示动作的关联

关联应该描述应用领域的结构化特征，而不是[临时性事件](#)。但有时候，表达成动作的需求隐含着下层的结构化关系，应该相应地重新处理。银行网络管理示例中，[ATM接受现金卡](#)描述ATM和客户之间的部分交互周期，而不是ATM和现金卡之间的永久关系。也可以清除[ATM与用户交互](#)。中心计算机清除银行的交易信息描述了一项动作，该动作暗含着结构化关系中心计算机与银行通信。

□三元关联尽量处理为二元关联

可以把3个或更多个类之间的大多数关联分解成二元关联或重新把它们描述成限定关联。如果三元关联中的某个术语纯粹是描述性的，没有自己的标识，那么这个术语就是二元关联上的属性。

关联“公司发工资给某人”可以重新描述成二元关联“公司雇用某人”，每一个公司-个人链接上都有工资值。

ATM示例中，“银行计算机处理帐户上的交易信息”可以拆分成“银行计算机处理交易”和“交易涉及帐户”。出纳员输入帐户的交易信息也可以进行类似地拆分。ATM机与中心计算机通信交易信息实际上是二元关联与中心计算机通信和交易信息在ATM机上输入。

□命名不当的关联

关联名称对正确理解关联很重要，应该谨慎地选择，抓住关联的本质内涵。

银行网络管理示例中，“银行计算机维护帐户”是一项动作声明；应描述为“银行持有帐户”。



■ (5) 定义关联终端名（角色）、限定关联、多重性

□ 在合适的地方增加关联终端名

同一个类的两个实例之间的关联需要用关联终端名来区分实例。例如，关联 Person Manager person 需要终端名 Boss 和 Worker。

□ 限定关联

通常，名称在某种上下文中确定对象，多数名称不是全局唯一的。上下文环境与名称一起唯一确定对象。限定符可以区分在关联中“多”方上的对象。

银行网络管理示例中，限定符 Bank Code 区分某个联盟中的不同银行。每张现金卡都需要银行代码，这样交易就可以直接导向相应的银行。

□ 指定多重性

不要投入太多时间完全弄清楚它，多重性在分析过程中经常会发生变化。挑战“一”方的多重性取值。

例如，关联一名经理管理多名雇员排除了出现矩阵管理或职责分离的雇员的可能性。对于“多”方的多重性取值要考虑是否需要限定符；也要问一问对象是否需要以某种方式排序。

■ (6) 寻找属性

- 属性是独立对象的数据特征，例如重量、速度或颜色等。

- 属性值不应该是对象；

- 属性通常与跟所有格短语的名词相对应，

例如“汽车的颜色”或“鼠标的位置”。形容词经常表示的是特定的枚举属性值，例如红（RED），打开（ON）或过期（EXPIRED）。与类和关联不同，属性更不可能在问题陈述中完整地描述。必须利用你在应用领域和真实世界中的知识来寻找它们。你也可以在相关系统的制品当中寻找属性。幸运的是，属性几乎不会影响问题的基本结构。

建议：不要把寻找属性的过程过分放大，只需要考虑那些与应用直接相关的属性。首先获取最重要的属性，可以在以后的设计过程中逐步添加细节。

■ (7) 删除那些不必要和不正确的属性

具有下列特征的候选项，一般是不必要的或不正确的属性

□到底是属性还是对象

如果元素的独立存在性很重要，那么它就是对象。例如，boss指的是一个类，salary（薪水）是一个属性。这种差异往往要视应用场景而定。例如，在邮件列表中，city可能会看成属性；而在人口普查中，City会是有着许多属性及其自身关系的一个类。在给定应用中有着自身关系的元素是一个类

□到底是属性还是限定符

如果属性值依赖于特定的上下文，那就要把属性重新描述为限定符号。例如，employee Number 不是一名拥有两项工作的人的唯一特性，它限定了关联 Company employs Person。

□到底是属性还是标识符

OO语言合并了对象标识符的概念，用于无歧义地引用对象。不要包含那些存在的唯一目的只是用于确定对象的属性，因为在类模型中对象标识符是隐含的。只列举那些在应用领域中存在的属性。

例如，account Code是一个真正的属性；Banks分配account Code，客户查看它们。与此相反，不应该把内部的transaction ID列为属性，尽管在实现过程中这样做会很方便。

□关联上的属性

如果某个值依附于链接而存在，那么此特征就是关联的属性，而不是相关类的属性。

在多对多关联上，属性通常是很明显的；由于多重性，它们不会依附于哪一个类。例如，在Person和Club之间的关联中，属性membership date属于关联，因为一个人可以属于多间俱乐部，一家俱乐部也可以有多名成员。

在一对多关联上，属性更加不明显，因为它们可以依附于“多”侧那个类，而不会丢失任何信息。抵制住把它们依附于类这种欲望，因为多重性发生变化，它们就会变得不合法。在一对一关联上，属性也是不十分明显。

□内部取值

如果属性描述的是在外部不可见的一个对象的内部状态，那么就把它从分析过程中删除。

□细节

忽略那些不太可能会影响多数操作的次要属性。

□与其他属性不相关的属性

看上去与所有其他的属性完全不同或无关的属性可能意味着类应该分割成两个不同的类。

类应该是简单且一致的。

不专一的类经常产生于分析过程中不成熟的实现决策。

□布尔属性

重新考虑所有的布尔属性。常常可以拓宽布尔属性，把它重新声明为枚举

■ (8) 通过泛化或集成来组织类之间的结构

泛化（继承）指父类与一个或多个子类之间的关系。

泛化利用类的相似性和差异性来组织类。同样描述对象的结构，父类拥有公共属性、操作和关联，子类增加了特定的属性、操作和关联。

每个子类继承了其父类的特征。

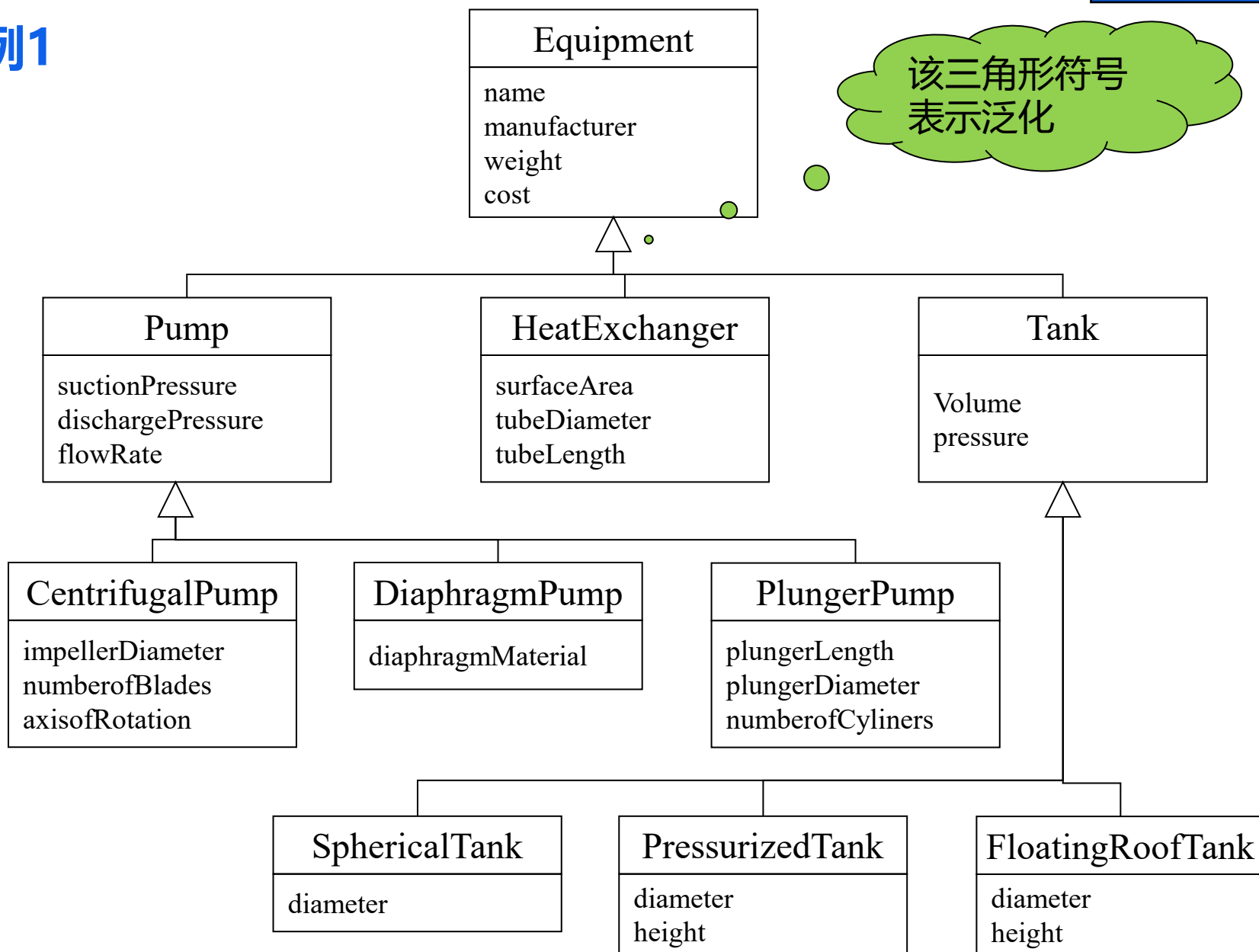
泛化有时被称为“ is-a ” 关系

泛化可以将类分级，每个子类有一个直接父类。泛化可以有多层。



1 建立类模型

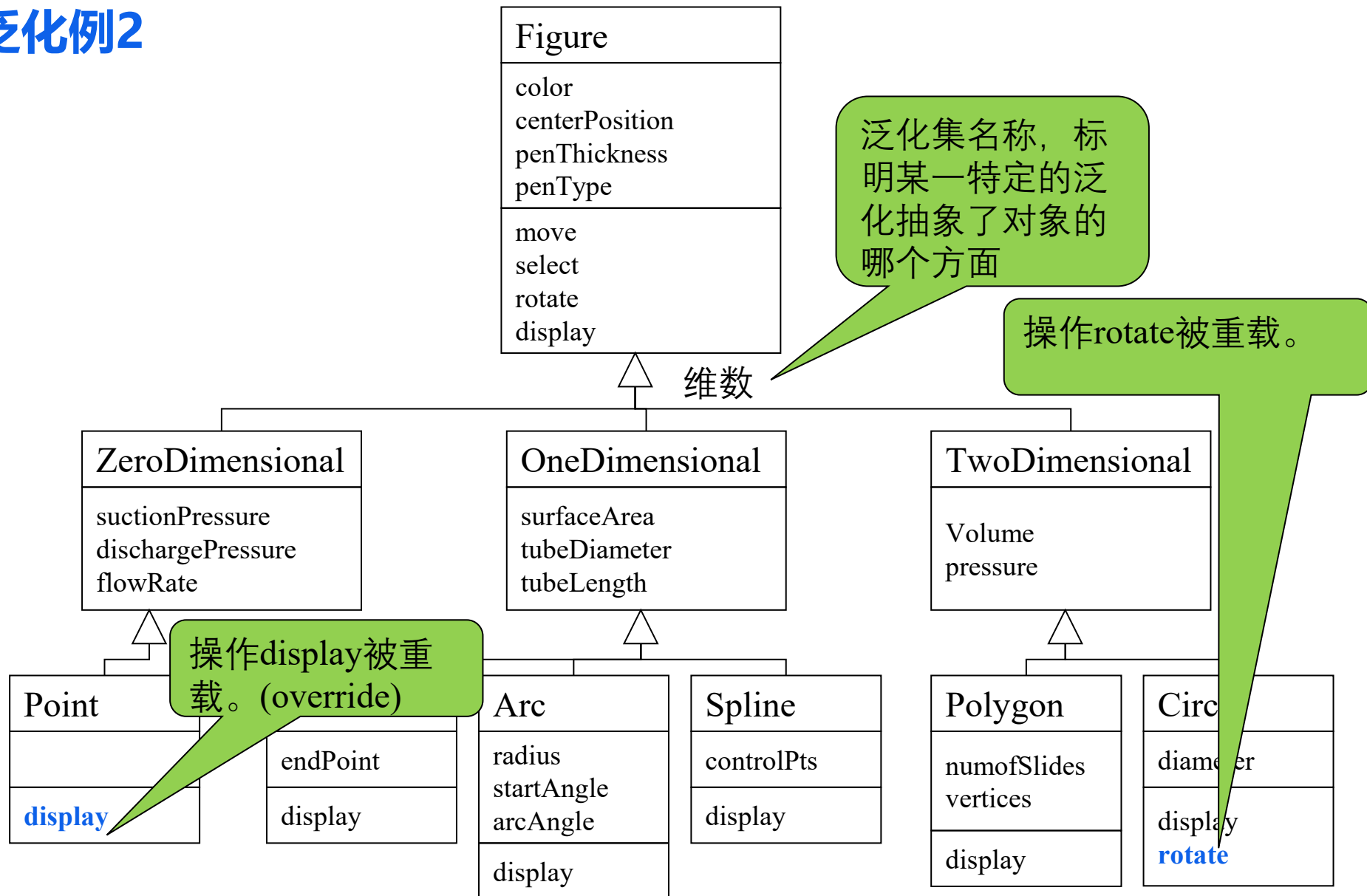
泛化例1





1 建立类模型

泛化例2



□ 利用泛化来组织类结构

通过使用继承共享公共结构来组织类，继承可以在两个方向上添加：

◆ 将现有类的公共方面泛化为父类（自下而上）

通过搜索有着相似属性、关联和操作的类，就可以自下而上地发现继承。

对于每种泛化，定义一个父类来共享公共的特征。必须要略微重定义一些属性或类来做好配合。这是可以接受的，如果无法配合也不要过分勉强；那样就会出现错误的泛化。

一些泛化会基于现实世界中的现有分类方法而突出出来，只要有可能就使用现有的概念。对称性会显示出一些遗漏的类。

ATM示例。除了来源不同之外，远程事务和出纳站事务很相似，可以由事务泛化而来。另一方面，对于ATM示例来说，中心计算机和分行计算

◆ 将现有的类特化为多个子类（自上而下）

自上而下的特化经常可以从应用领域明显地看出来，可以在类名中寻找由不同的形容词组成的名词短语，如：

- ◆ 荧光灯（fluorescent lamp）、白炽灯（incandescent lamp）；

- ◆ 固定菜单（fixed menu），弹出菜单（pop-up menu）和滑动菜单（sliding menu）。

要避免过度地细化，如果提出的规约与现有的类不相符，那么现有的类就有可能是表达的不合理。

□ 泛化对枚举

在应用领域中，被枚举的子情形是特化最常见的来源。注意到存在一组枚举子情形，一般已经足够了，并不需要实际列出它们。

例如，ATM帐户可以细化成Checking Account（活期存款账户）和Savings Account（储蓄账户）。毫无疑问，在一些银行应用中，这是很有用的，但这种差异不会影响ATM应用程序内部的行为；Account可以有一个简单的属性---- type.

□ 多重继承

可以用多重继承来加大共享的程度，但只在有必要的时候才这么做，因为那样会同时增加概念和实现的复杂性。

□ 相似的关联

当同一个关联名称出现多次，其内在含义却相同时，就要试着泛化关联类。有时候，除了关联之外，类与类之间没有共同点，但更常见的是，你会发现曾经忽视的那些下层的普遍性。

ATM示例：

- ◆ 事务可以有出纳站事务和ATM输入事务；
- ◆ 录入站泛化为出纳站和ATM。

□ 聚合 (aggregation)

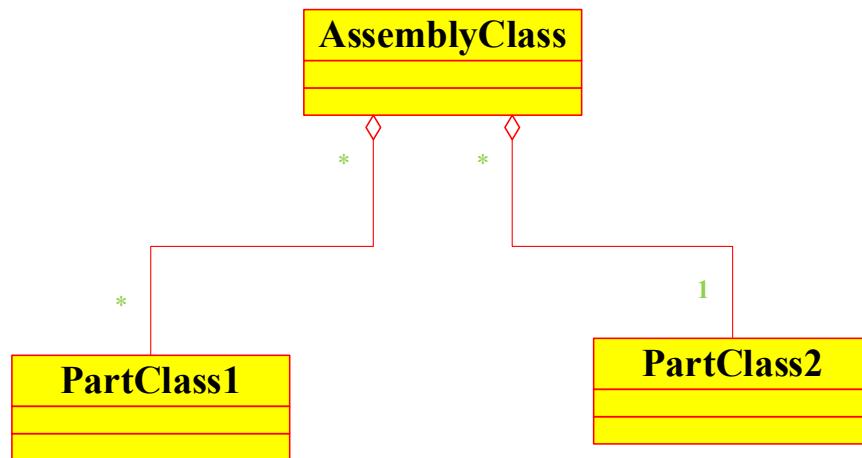
一个类有时是由几个部分类组成的，这种特殊的整体与部分的关系称为聚合（或聚集）。

聚合关系中整体和部分不是强依赖的，即使整体不存在了，部分依然存在。

聚合关系可以构成一个层次结构，整体类位于结构的最顶端，以下依次是各个部分类。

在UML中，整体类和部分类之间用带空心的菱形箭头线连接，箭头指向整体。

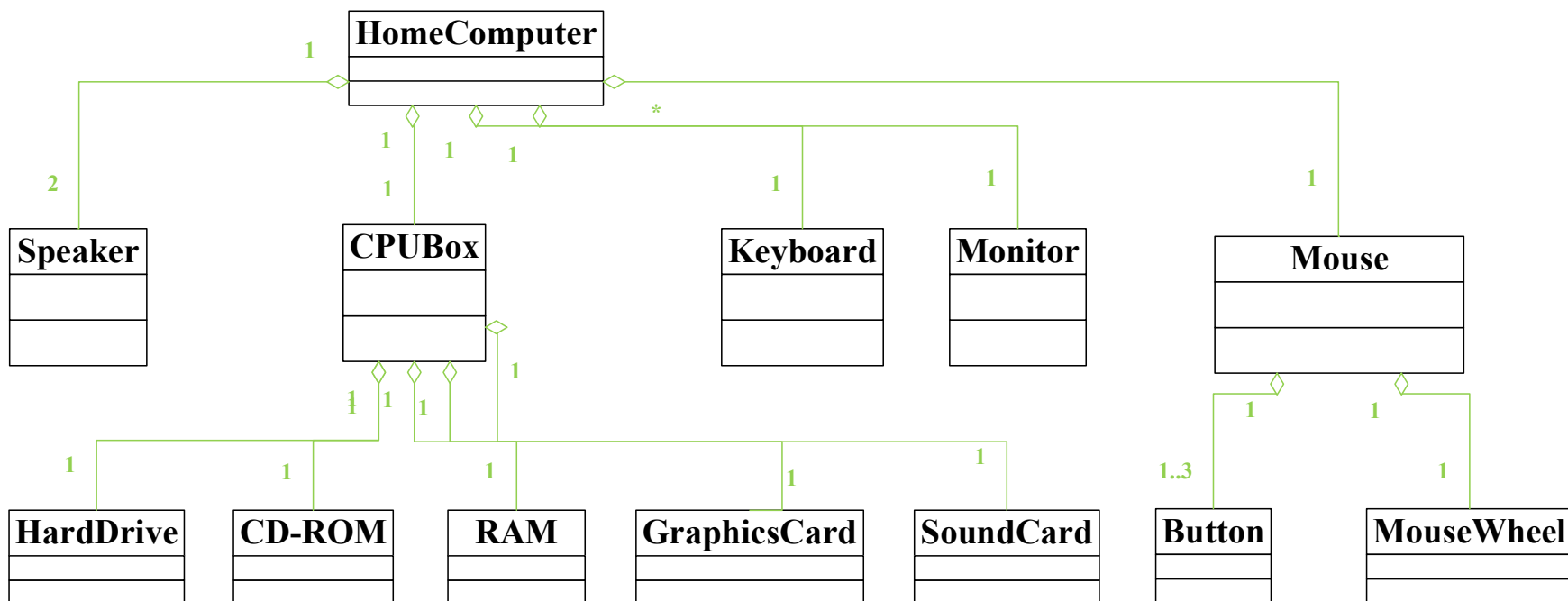
有时一个聚合体可能有多种部分体组成，这些部分体之间是“或”关系。这种“或”关系通过约束来表达。





1 建立类模型

家用计算机—聚合例子：



```
classDiagram
    class Meal {
    }
    class Soup {
    }
    class Salad {
    }
    class MainCourse {
    }
    class Dessert {
    }
    Meal "1" *-- "1" Soup
    Meal "1" *-- "1" Salad
    Meal "1" *-- "1" MainCourse
    Meal "1" *-- "1" Dessert
    Meal .. "1" Salad : {or}
```

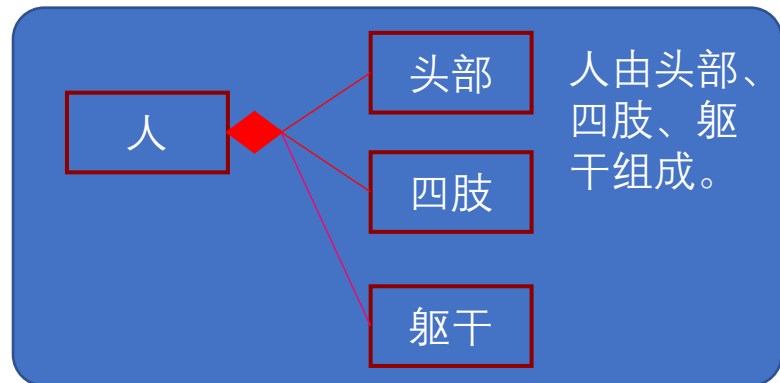
组合

组合（composition）关系用于类图，特别用于表示实体对象关系，表达整体拥有部分的语义，而且每个部分体只能属于一个整体。

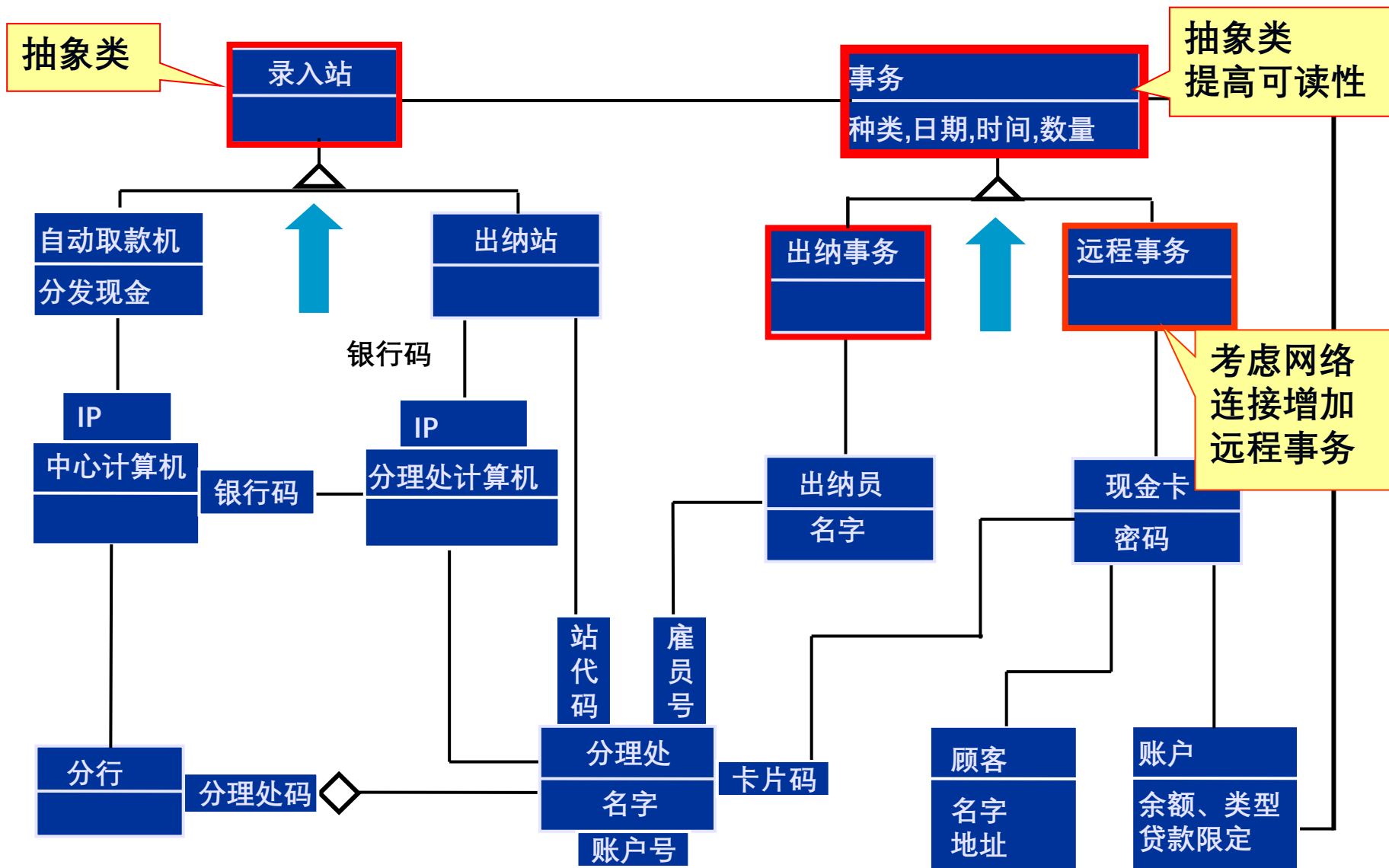
例如：母公司拥有许多子公司；

组合（composition）是一种强依赖的聚合关系，如果整体不存在了，则部分也将消亡。例如母公司解体，子公司也将不存在。

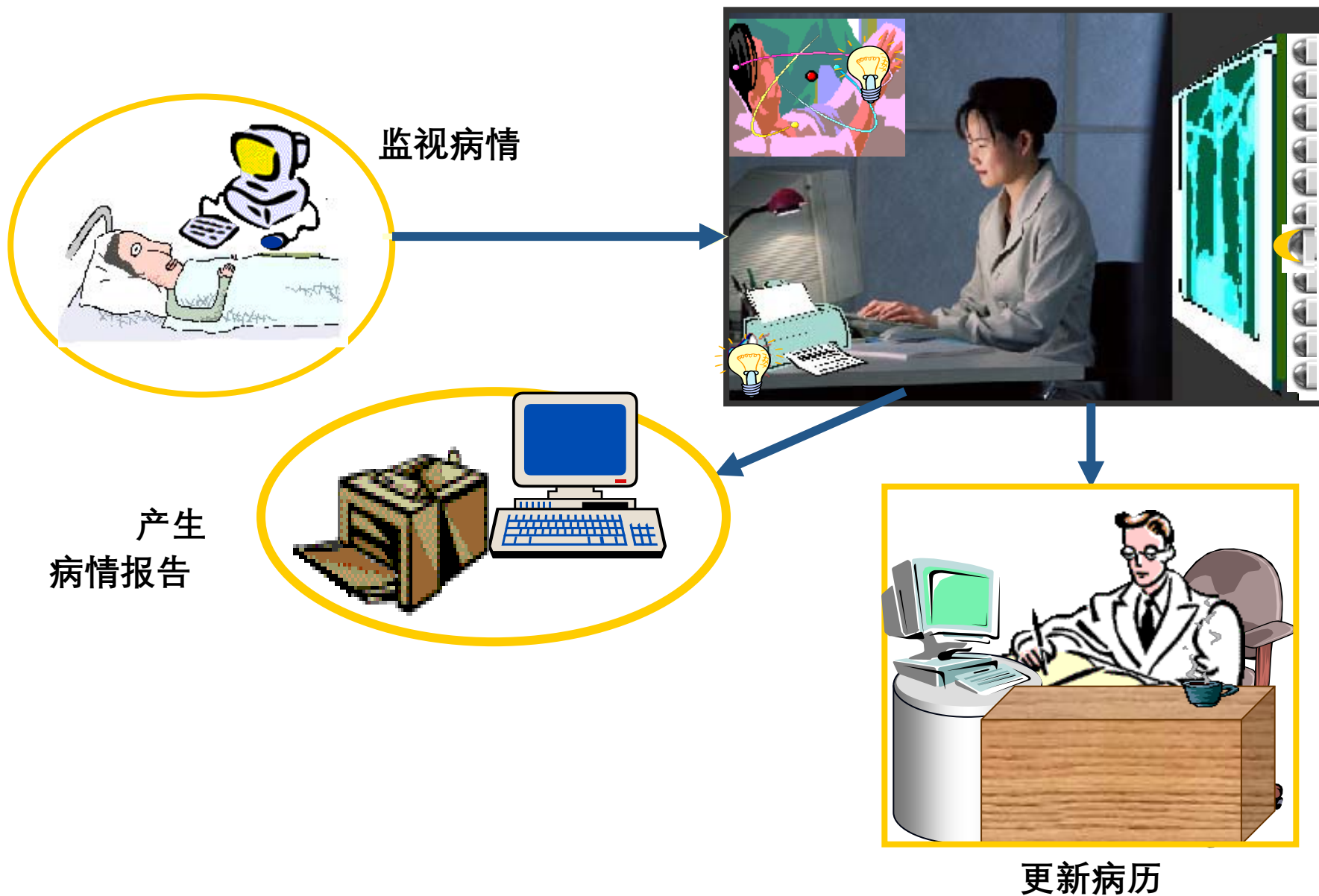
在UML中组合的表示法与聚合类似，区别在于菱形箭头是实心的。



1.3类建模实例：ATM



1.3类建模实例：医院病房监护系统



画出例中医院病房监护系统的类图

分析：为了对危重病人进行实时监护，随时了解病人病情，及时进行处理，建立病房监护系统。

- ❑ 病症监视器放置在每个病床，通过网络将病人的组合病症信号实时传送到中央监护系统进行分析处理。
- ❑ 在中心值班室里，值班护士使用中央监护系统对病员的情况进行监控，监护系统实时地将病人的病症信号与标准的病诊信号进行比较分析，当病症出现异常时，系统会立即自动报警，并打印病情报告和更新病历。
- ❑ 系统根据医生的要求随时打印病人的病情报告，系统定期自动更新病历。

1.3类建模实例：医院病房监护系统

根据名词识别法和实体识别法,确定了以下初始类:

监视器	网络	值班护士	医生	病人	病症监视
中央监护系统	报警信号	标准病症信号库	病历库		
病人病症信号	病情报告	标准病症信号	病历		
中心值班室	处理	组合病症信号			

分析确定类:

- ① 去掉冗余类: 保留最具有描述能力的类。
- ② 去掉不相干的类: 删除与问题无关或关系不大的类。
- ③ 删除模糊的类或性质独立性不强的类: 有些初始类边界定义不确切, 或范围太广, 应该删除。
- ④ 所描述的是属性或操作, 不适宜作为对象类, 所描述的只是实现过程中的暂时的对象, 应删去。

1、识别系统的类

通过名词识别法和系统实体识别法等方法可以确定系统的12个类：

值班护士 医生 病人 病症监视 中央监护系统 报警信号 标准病症信号库 病历库 病人病症信号 病情报告 病历 标准病症信号

值班护士	医生	病人	病症监视
用户名 密码	用户名 密码	姓名 性别 年龄 病症	采集频率 病症信号
查看病情报告 () 打印病情报告 ()	查看病情报告 () 要求打印病情报告 () 查看病历 () 要求打印病历 ()	提供病症信号 ()	格式化信号数据 () 采集信号 () 信号组合 ()

1.3类建模实例：医院病房监护系统

中央监护系统	报警信号	标准病症信号库	病历库
输入	声音	类型	类型
输出	灯光	大小	大小
	文字	容量	容量
分解信号 ()	报警 ()	提供标准信号 ()	生成病历 ()
比较信号 ()	数模转换 ()		更新病历 ()
报警 ()			查看病历 ()
数据格式化 ()			打印病历 ()

病人病症信号	病情报告	病历	标准病症信号
脉搏	标题	格式	脉搏
血压	格式	病人基本情况	血压
体温		打印时间	体温
生成病症信号 ()	生成病情报告 ()	生成病历 ()	生成标准信号 ()
	查看病情报告 ()	查看病历 ()	
	打印病情报告 ()	打印病历 ()	

2. 确定关联：

使用下列标准去掉不必要和不正确的关联：

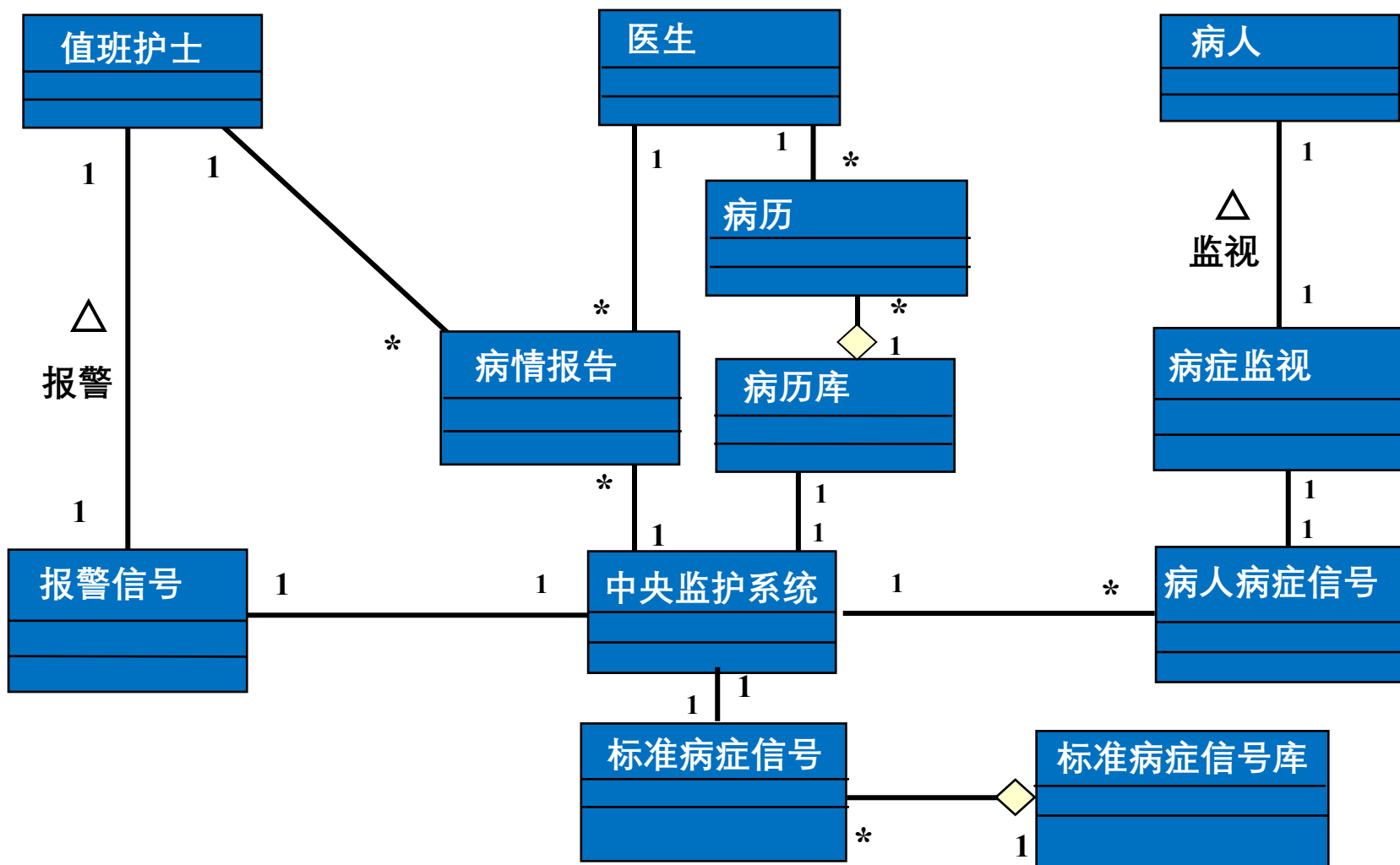
(1) 若某个类已被删除，那么与它有关的关联也必须删除或者用其他类来重新表述。

(2) 不相干的关联或实现阶段的关联。删除所有问题域之外的关联或涉及实现结构中的关联。

(3) 动作。关联应描述应用域的结构性质而不是瞬时效件。

(4) 派生关联，省略那些可以用其他关联来定义的关联,因为这种关联是冗余的。

3.画出医院病房监护系统类图





02

领域状态模型

一些领域对象在其生存期内会经历截然不同的状态。在这些不同的状态中，属性值上可能有不同的约束，会有不同的关联或多重性，会触发不同的操作，也会有不同的操作行为，等等。

状态图描述了对象可能假定的不同状态，不同状态中对象的属性和约束以及引发对象状态变化的事件。

多数领域类不需要状态图，可以通过一系列操作来充分地描述。但是，对于少数没有展示出不同状态的类，状态模型也有助于理解它们的行为。

构造领域状态模型的基本步骤：

- ◆ 首先都要用关键状态确定领域类，并记录每一个类的状态。
- ◆ 然后确定引发对象状态变化的事件。给定状态和事件，就可以构造出受影响对象的状态图。
- ◆ 最后，评价状态图，确保它们的完整性和正确性。



构造领域状态模型的基本步骤：

- (1) 确定具有状态的类
- (2) 寻找状态
- (3) 寻找事件
- (4) 构建状态图
- (5) 评价状态图

(1) 确定具有状态的类

- 具有清晰、有序的状态的类或实体最适合使用状态图进一步建模。
- 寻找那些有着不同生命周期的领域类。在对象的生命周期中，确定重要的状态。
- 寻找那些可以由递进历史所刻画类（例如：人）或者展示出循环行为的类（例如：电梯控制类）。在对象的生命周期中，确定重要的状态。

例如，某学报上的一篇科技论文可以经历状态指派编辑（editor assigned）、邀请审稿人（reviewer invited）、在审（under review）、编辑处理（with editor）等，然后到大修（major revision）、小修（minor revision）、接受（accepted）或退回（Rejected）的变化。这个过程中可能会有一些循环，例如审稿人意见不一致，编辑需要邀请新的审稿人，重新走审稿流程，但基本来说这个对象的生命还是递进的。

例如，某航空公司的飞机会经历下面的状态循环：维修（Maintenance）、装货（Loading）、飞行（Flying）和卸货（Unloading）。

并不是每一种状态都有一个循环，有些状态只会发生一次，而且可能还会出现其他状态。ATM示例。Account（账户）是一个重要的业务概念，相应的ATM行为也要取决于Account的状态。Account的生命周期混合了递进和循环式的问题状态。其他的ATM类并没有重要的领域状态模型。

(2) 寻找状态

- 列举每个类的状态，给每个状态赋予一个有意义的名称。
- 刻画每个类中的对象，包括对象的属性值、对象参与的关联以及关联的多重性、只在某些状态下有意义的属性和关联等等；
- 名称尽量直接描述状态，而不是表明状态来源。
- 状态应该以行为、属性或关联上的本质差异为基础，而不要专注于状态之间的微妙变化，特别是像大、中、小这样的量化差异。
- 在检查事件之前不必先确定所有的状态。通过浏览事件并参看状态之间的转换，遗漏的状态就会变得很清晰。

ATM示例。Account有一些状态：

- Normal（准备就绪，等待正常访问）
- Closed（客户结束帐户，但银行记录中仍有文件）
- Overdrawn（客户取款额度超过了帐户余额）
- Suspended（由于某个原因造成帐户访问受阻）

(3) 寻找事件

针对某一类的不同状态，寻找引发状态间转换的事件。

想一想有哪些激励会引起状态发生变化。

- 在许多情形下，可以把事件视为do活动完成的触发。

例如，如果一篇科技论文处在状态Under review下，那么当论文的评判意见下达时这个状态就会终止。这里，评判意见可以是正面（接受论文）或负面的（退回论文）。在完成do活动时，经常还会出现其他的可能性，并可能在未来会增加进来，例如，需要文章进行修订（major revision），并修改后再审等。

- 可以考虑让对象进入特定的状态，借此来寻找其他事件

例如：

- ◆ 打电话时拿起电话听筒，就会进入Dialing状态。
- ◆ 在电话接通以后，就会进入Ringing状态。

在状态内部还会发生一些事件，它们不会引发转换。

对于领域状态模型，应该关注那些引发状态间转换的事件。当发现了一个事件时，要以参数列表的形式来捕获它所传递的所有信息。

(4) 构建状态图

构建状态图应注意的问题:

- 注意每一个事件所应有的状态。
- 增加转换, 以显示对象处在特定状态时, 由出现的事件所引发的状态变化。
- 如果一个事件终止了某状态, 它通常在那个状态和另一个状态之间有单个转换。
- 如果一个事件引发了一个目标状态, 那么要考虑该事件可能发生的地点, 并添加从那些状态到目标状态的转换。
- 如果一个事件在不同的状态中有不同的作用, 那么就为每一个状态增加一次转换。
- 在将转换确认下来以后, 就要考虑在无转换发生的状态中事件的含义。
 - 该事件是被忽略了吗? 如果是, 那么一切都好。
 - 它是表示错误吗? 如果是, 那么就增加到错误状态的转换。
 - 它会有一些被你遗忘了的效应? 如果是, 那么就增加另一项转换。有时候, 还会发现一些新的状态。

■绘制状态图

绘制状态图的理想步骤是：**寻找**主要的状态，确定状态之间的**转换**，**细化**状态内的活动与转换，用复合状态来展开细节

➤寻找主要状态：

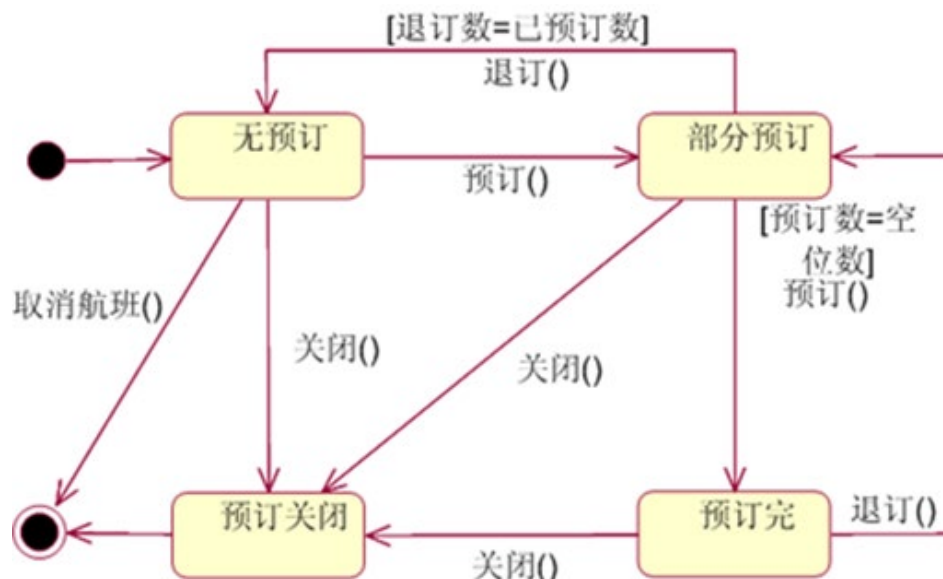
对于航班机票预订系统而言，显然包括的状态主要有：

- 在刚确定飞机计划时，显然是没有任何预订的，并且在有人预订机票之前都将处于这种“无预订”状态
- 对订座而言显然有“部分预订”和“预订完”两种状态
- 而当航班快要起飞时，显然要“预订关闭”

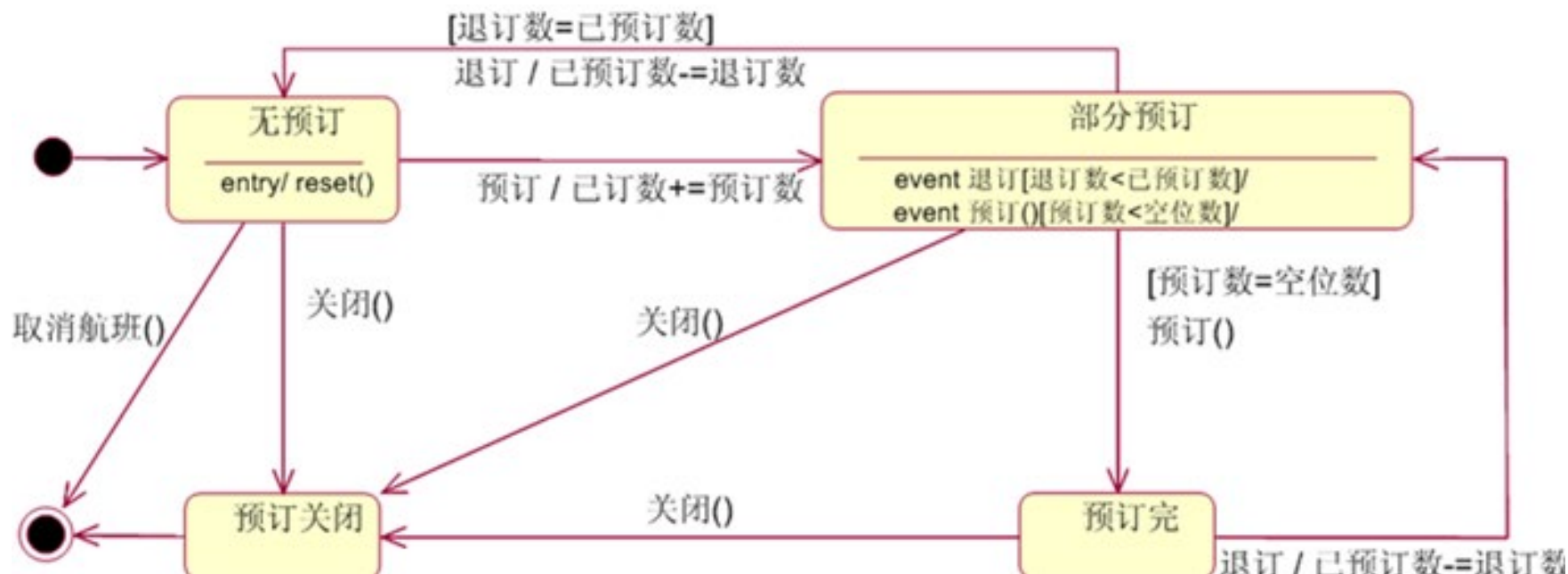
总结一下，主要有四种状态：无预订、部分预订、预订完以及预订关闭

➤ 确定状态间转换：表格横向是转出，表格纵向是转入

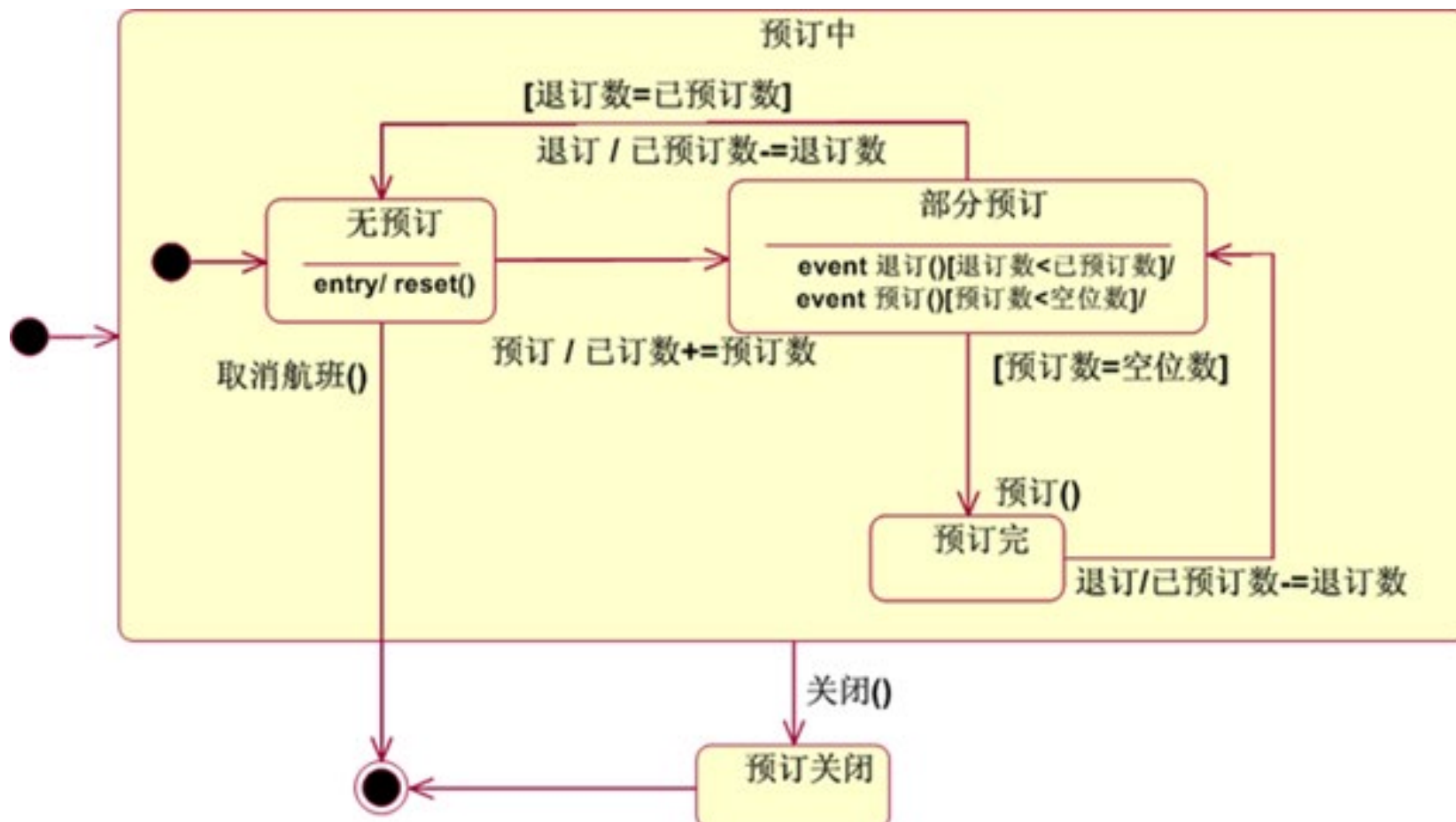
源目标	无预订	部分预订	预订完	预订关闭
无预订		预订 ()	不直接转换	关闭 ()
部分预订	退订 (), 使预订人 =0		预订 (), 无空座	关闭 ()
预订完	不直接转换	退订 ()		关闭 ()
预订关闭	无转换	无转换	无转换	



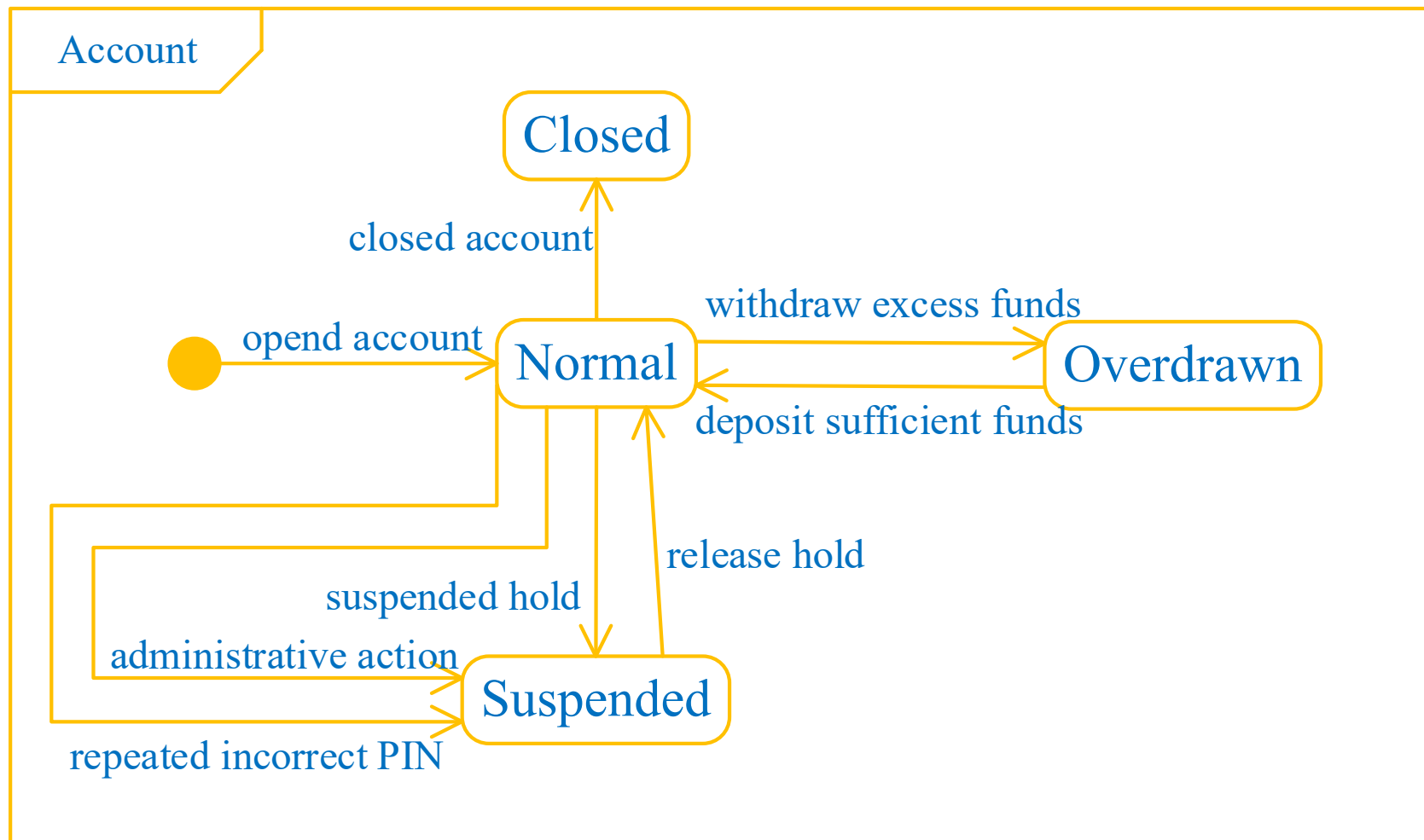
► 细化状态内的活动与转换



➤ 使用复合状态



ATM示例：类Account的领域状态模型



(5) 评价状态图

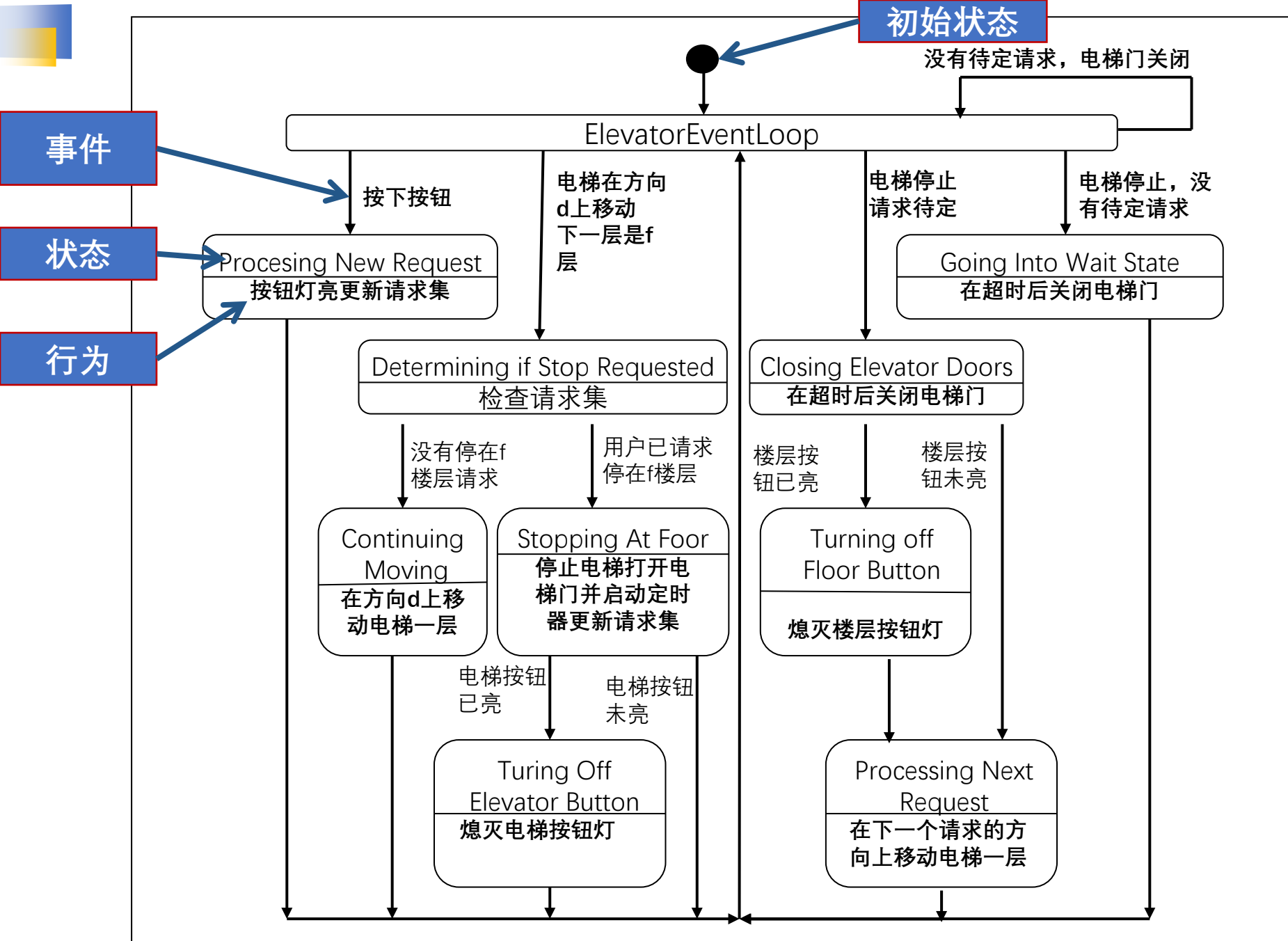
检查每一个状态模型。

- 所有的状态都连接起来了吗？要特别注意贯穿状态的路径。
- 如果状态模型表示的是一种递进式的话，那么存在一条从初始状态到最终状态的路径吗？
- 如果它表示的是一种循环式的话，那么有没有表示主循环？是不是存在可以终止循环的死状态呢？
- 用领域知识来寻找遗漏的路径。
- 有时候，漏掉的路径表明有漏掉的状态。如果状态模型完成了，它应该可以准确地表示类的生命周期。

ATM示例。Account的状态模型非常简单，对它也很满意。要构造更深层次的模型，就需要大量银行业的知识。

从电梯的使用场景中，可以推导出电梯控制类的相关状态：

- ◆ 电梯的空闲状态，由系统自动循环检查请求集— Elevator Event Loop
- ◆ 如果在电梯空闲状态下，按下按钮，电梯要处理请求 — Processing New Request
- ◆ 电梯在运行过程中，需要检查在下一楼层是否有停止请求 — Determining If Stop Requested
- ◆ 如果没有的话，继续移动 — Continuing Moving
- ◆ 如果有的话，在电梯门处停止并打开电梯门 — Stopping At Floor
- ◆ 如果有电梯按钮灯亮，需要关闭电梯按钮灯 — Turning off Elevator Button
- ◆ 一般在开门后电梯会启动定时器，超时后关闭电梯门 — Closing Elevator Door
- ◆ 如果楼层灯亮，关闭电梯门后需要熄灭楼层灯— Turning Off Floor Button
- ◆ 处理下一个请求— Processing Next Request
- ◆ 如果电梯停止后没有待定请求，则需要在超时后关闭电梯门— Going Into Wait State



Elevator Controller Class 状态图的第一次迭代



03

领域交互模型

对于领域分析来说，交互模型不太重要。在领域分析的过程中，重点在与关键的概念和深层的结构化关系上，而不在用户视图上。但是交互模型在应用建模的时候是一个重要的方面，会应用分析里讨论这个主题。



04

将分析迭代

大多数分析模型不可能只一遍就完成。问题陈述常常要包含一些迂回的内容，大多数应用也不能以完全线性的方式完成，因为问题的不同部分会有交互。要理解问题的全部含义，就必须迭代地进行分析，完成模型的第一次逼近，然后随着理解程度的增加，不断的迭代分析。在分析和设计之间没有严格的界线，因此不要过于考虑界线，要由应用领域专家和用户来验证最终的分析结果。

下面是一些关于细化分析模型的经验之谈：

在模型内部或者模型之间，整个分析模型都可能显示出不一致和不平衡的地方。要迭代不同的部分，以生成一种更加整洁和一致的模型。尽量细化类以增加共享性并完善结构。增加第一遍中有待完善的细节。

有时候，还会感到一些已定义类很笨拙，不怎么到位。要仔细地重新检查它们；可能是概念有错误。有时候，随着对问题更深入的理解，会需要在模型当中重新组织结构。现在做要比将来做容易一些，因此不要规避变化，因为现在已经有有一个模型，当有许多制品看起来相似但并不匹配时，那么可能是漏掉了或者误用了一种更为通用的概念。要警惕泛化，防止在错误的层面上对它们进行分解。

一种常见的难题是在逻辑上拥有两种不同侧面的实体对象。每一方面都应该使用一重不同的对象来建模。出现这种问题的一种标志是，类匹配得不好，看起来似乎它有两套无关的属性、关联和操作。

另一种标志是包含异常和许多特例，并且缺少期望的对称性。考虑重新构造模型的结构，以更好地在结构内部捕获约束。

要预防在模型中随意编纂业务实践的做法。软件应该提供业务操作便利，而不能抑制合理的变化。可以引入抽象概念来增加业务灵活性，同时也不会把模型弄的很复杂。

删除那些初看起来有用、实则多余的类或关联。分析中的两个类常常可以合并起来，因为它们的差异并没有以一种有意义的方式影响到模型的其余部分。随着分析过程的进行，模型会有一种增长的趋势。这值得注意，因为随着模型越来越大，开发工作量也会逐步上升。仔细观察模型，寻找那些可以删除的次要概念以及简化模型的抽象。

对于优秀的模型，能感觉出来其正确性，它们不会有多余的细节。如果模型不够完美，也不必担心；即使是优秀的模型也常常还会有一些小地方，虽然设计得很充分，但感觉总是不好。完美的系统是很难构建的。

重述需求

在分析完成了以后，模型就会成为需求的基础，它界定了未来论述的范围。

大多数真实的需求将会是模型的一部分。

还会有一些性能约束；这些约束应该与优化标准一起清晰地描述出来。

应该与请求者一起验证最终模型。在分析过程中，会出现一些不正确和不实际的需求；确认在需求上做出的修改。业务专家也应该验证分析模型，以确保模型正确地建模了真实世界。分析模型是一种与业务专家（他们不是计算机专家）交流的有效的方法。

最终经过验证的分析模型是系统架构、分析和实现的基础。应该修改原始的问题陈述，加入在分析过程中完成的修改以及自己的理解。



THANK YOU

浙江大学机械工程学院航空制造工程研究所
王青 李江雄